

CHAPTER 3

Pointers and Array-Based Lists

IN THIS CHAPTER, YOU WILL:

- Learn about the pointer data type and pointer variables
- Explore how to declare and manipulate pointer variables
- Learn about the address of operator and dereferencing operator
- Discover dynamic variables
- Examine how to use the `new` and `delete` operators to manipulate dynamic variables
- Learn about pointer arithmetic
- Discover dynamic arrays
- Become aware of the shallow and deep copies of data
- Discover the peculiarities of classes with pointer data members
- Explore how dynamic arrays are used to process lists
- Learn about virtual functions
- Become aware of abstract classes

The data types in C++ are classified into three categories: simple, structured, and pointers. Until now, you have worked with only the first two data types. This chapter discusses the third data type: the pointer data type. You first learn how to declare pointer variables (or pointers, for short) and manipulate the data to which they point. Later, you use these concepts when you study dynamic arrays and linked lists. Linked lists are discussed in Chapter 5.

The Pointer Data Type and Pointer Variables

The values belonging to pointer data types are the memory addresses of your computer. However, there is no name associated with the pointer data type in C++. Because the domain, (that is, the values of a pointer data type), consists of addresses (memory locations), a pointer variable is a variable whose content is an address, that is, a memory location.

Pointer variable: A variable whose content is an address (that is, a memory address).

Declaring Pointer Variables

The value of a pointer variable is an address. That is, the value refers to another memory space. The data is typically stored in this memory space. Therefore, when you declare a pointer variable, you also specify the data type of the value to be stored in the memory location to which the pointer variable points.

In C++, you declare a pointer variable by using the asterisk symbol (`*`) between the data type and the variable name. The general syntax to declare a pointer variable is as follows:

```
dataType *identifier;
```

As an example, consider the following statements:

```
int *p;
char *ch;
```

In these statements, both `p` and `ch` are pointer variables. The content of `p` (when properly assigned) points to a memory location of type `int`, and the content of `ch` points to a memory location of type `char`. Usually `p` is called a pointer variable of type `int`, and `ch` is called a pointer variable of type `char`.

Before discussing how pointers work, let us make the following observations. The following statements that declare `p` to be a pointer variable of type `int` are equivalent:

```
int *p;
int*  p;
int * p;
```

Thus, the character * can appear anywhere between the data type name and the variable name.

Now consider the following statement:

```
int* p, q;
```

In this statement, only p is a pointer variable, not q. Here q is an int variable. To avoid confusion, we prefer to attach the character * to the variable name. So the preceding statement is written as follows:

```
int *p, q;
```

Of course, the statement

```
int *p, *q;
```

declares both p and q to be pointer variables of type int.

Now that you know how to declare pointers, next we discuss how to make a pointer point to a memory space and how to manipulate the data stored in these memory locations.

Because the value of a pointer is a memory address, a pointer can store the address of a memory space of the designated type. For example, if p is a pointer of type int, p can store the address of any memory space of type int. C++ provides two operators—the address of operator (&) and the dereferencing operator (*)—to work with pointers. The next two sections describe these operators.

Address of Operator (&)

In C++, the ampersand, &, called the **address of operator**, is a unary operator that returns the address of its operand. For example, given the statements

```
int x;
int *p;
```

the statement

```
p = &x;
```

assigns the address of x to p. That is, x and the value of p refer to the same memory location.

Dereferencing Operator (*)

The previous chapters used the asterisk character, *, as the binary multiplication operator. C++ also uses * as a unary operator. When *, commonly referred to as the **dereferencing operator** or **indirection operator**, is used as a unary operator, * refers to the

object to which the operand of the * (that is, the pointer) points. For example, given the statements

```
int x = 25;
int *p;
p = &x;   //store the address of x in p
```

the statement

```
cout << *p << endl;
```

prints the value stored in the memory space to which **p** points, which is the value of **x**. Also, the statement

```
*p = 55;
```

stores 55 in the memory location to which **p** points—that is, 55 is stored in **x**.

Example 3-1 shows how a pointer variable works.

EXAMPLE 3-1

Let us consider the following statements:

```
int *p;
int num;
```

In these statements, **p** is a pointer variable of type **int** and **num** is a variable of type **int**. Let us assume that memory location 1200 is allocated for **p** and memory location 1800 is allocated for **num**. (See Figure 3-1.)

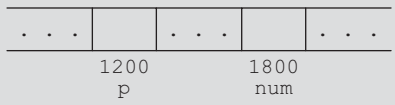
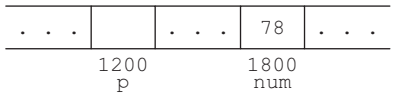
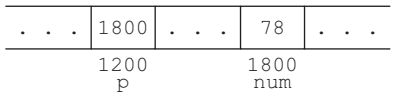
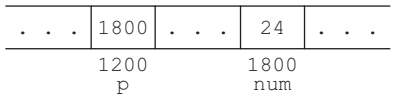


FIGURE 3-1 Variables p and num

Consider the following statements:

1. `num = 78;`
2. `p = #`
3. `*p = 24;`

The following shows the values of the variables after the execution of each statement.

After statement	Values of the variables	Explanation
1		The statement `num = 78;` stores `78` into `num`.
2		The statement `p = #` stores the address of `num`, which is `1800`, into `p`.
3		The statement `*p = 24;` stores `24` into the memory location to which `p` points. Because the value of `p` is `1800`, statement 3 stores `24` into memory location `1800`. Note that the value of `num` is also changed.

Let us summarize the preceding discussion.

1. A declaration such as `int *p;` allocates memory for `p` only, not for `*p`. Later, you learn how to allocate memory for `*p`.
2. The content of `p` points only to a memory location of type `int`.
3. `&p`, `p`, and `*p` all have different meanings.
4. `&p` means the address of `p`—that is, `1200` (as shown in Figure 3-1).
5. `p` means the content of `p`, which is `1800`, after the statement `p = #` executes.
6. `*p` means the content of the memory location to which `p` points. Note that the value of `*p` is `78` after the statement `p = #` executes; the value of `*p` is `24` after the statement `*p = 24;` executes.

The program in Example 3-2 further illustrates how a pointer variable works.

EXAMPLE 3-2

```
//*********************************************************
// Author: D.S. Malik
//
// This program illustrates how a pointer variable works.
//*********************************************************

#include <iostream>                                    //Line 1

using namespace std;                                   //Line 2
```

```
int main()                                                  //Line 3
{                                                           //Line 4
    int *p;                                                 //Line 5
    int num1 = 5;                                           //Line 6
    int num2 = 8;                                           //Line 7

    p = &num1; //store the address of num1 into p; Line 8

    cout << "Line 9: &num1 = " << &num1
         << ", p = " << p << endl;                          //Line 9
    cout << "Line 10: num1 = " << num1
         << ", *p = " << *p << endl;                        //Line 10

    *p = 10;                                                //Line 11
    cout << "Line 12: num1 = " << num1
         << ", *p = " << *p << endl << endl;                //Line 12

    p = &num2; //store the address of num2 into p; Line 13

    cout << "Line 14: &num2 = " << &num2
         << ", p = " << p << endl;                          //Line 14
    cout << "Line 15: num2 = " << num2
         << ", *p = " << *p << endl;                        //Line 15

    *p = 2 * (*p);                                          //Line 16
    cout << "Line 17: num2 = " << num2
         << ", *p = " << *p << endl;                        //Line 17

    return 0;                                               //Line 18
}                                                           //Line 19
```

Sample Run:

```
Line 9: &num1 = 0012FF54, p = 0012FF54
Line 10: num1 = 5, *p = 5
Line 12: num1 = 10, *p = 10

Line 14: &num2 = 0012FF48, p = 0012FF48
Line 15: num2 = 8, *p = 8
Line 17: num2 = 16, *p = 16
```

For the most part, the preceding output is straightforward. Let us look at some of these statements. The statement in Line 8 stores the address of `num1` into `p`. The statement in Line 9 outputs the value of `&num1`, the address of `num1`, and the value of `p`. (Note that the values output by Line 9 are machine dependent. When you execute this program on your computer, you are likely to get different values of `&num1` and `p`.) The statement in Line 10 outputs the value of `num1` and `*p`. Because `p` points to the memory location of `num1`, `*p` outputs the value of this memory location, that is, of `num1`. The statement in Line 11 changes the value of `*p` to `10`. Because `p` points to the memory location `num1`, the value of `num1` is also changed. The statement in Line 12 outputs the value of `num1` and `*p`.

The statement in Line 13 stores the address of `num2` into `p`. So after the execution of this statement, `p` points to `num2`. So, any change that `*p` makes immediately changes the value

of num2. The statement in Line 14 outputs the address of num2 and the value of p. This statement in Line 16 multiplies the value of *p, which is the value of num2, by 2 and stores the new value into *p. This statement also changes the value of num2. The statement in Line 17 outputs the value of num2 and *p.

Pointers and Classes

Consider the following statements:

```
string *str;
str = new string;
*str = "Sunny Day";
```

The first statement declares str to be a pointer variable of type string. The second statement allocates memory of type string and stores the address of the allocated memory in str. The third statement stores the string "Sunny Day" in the memory to which str points. Now suppose that you want to use the string function length to find the length of the string "Sunny Day". The statement (*str).length() returns the length of the string. Note the parentheses around *str. The expression (*str).length() is a mixture of pointer dereferencing and the class component selection. In C++, the dot operator, ., has a higher precedence than the dereferencing operator, *. Let us elaborate on this a little more. In the expression (*str).length(), the operator * evaluates first, so the expression *str evaluates first. Because str is a pointer variable of type string, *str refers to a memory space of type string. Therefore, in the expression (*str).length(), the function length of the class string executes. Now consider the expression *str.length(). Let us see how this expression gets evaluated. Because . has a higher precedence than *, the expression str.length() evaluates first. The expression str.length() would result in a syntax error because str is *not* a string object, so it cannot use the function length of the class string.

As you can see, in the expression (*str).length(), the parentheses around *str are important. However, typos are unavoidable. Therefore, to simplify the accessing of class or struct components via a pointer, C++ provides another operator, called the **member access operator arrow**, ->. The operator -> consists of two consecutive symbols: a hyphen and the "greater than" symbol.

The syntax for accessing a class (struct) member using the operator -> is as follows:

```
pointerVariableName->classMemberName
```

Thus, the expression

```
(*str).length()
```

is equivalent to the expression

```
str->length()
```

Accessing `class` (`struct`) components via pointers using the operator `->` thus eliminates the use both of the parentheses and of the dereferencing operator. Because typos are unavoidable and missing parentheses can result in either an abnormal program termination or erroneous results, when accessing `class` (`struct`) components via pointers, this book uses the arrow notation.

Initializing Pointer Variables

Because C++ does not automatically initialize variables, pointer variables must be initialized if you do not want them to point to anything. Pointer variables are initialized using the constant value `0`, called the **null pointer**. Thus, the statement `p = 0;` stores the null pointer in `p`; that is, `p` points to nothing. Some programmers use the named constant `NULL` to initialize pointer variables. The following two statements are equivalent:

```
p = NULL;
p = 0;
```

The number `0` is the only number that can be directly assigned to a pointer variable.

Dynamic Variables

In the previous sections, you learned how to declare pointer variables, how to store the address of a variable into a pointer variable of the same type as the variable, and how to manipulate data using pointers. However, you learned how to use pointers to manipulate data only into memory spaces that were created using other variables. In other words, the pointers manipulated data into existing memory spaces. So what is the benefit to using pointers? You can access these memory spaces by working with the variables that were used to create them. In this section, you learn about the power behind pointers. In particular, you learn how to allocate and deallocate memory during program execution using pointers.

Variables that are created during program execution are called **dynamic variables**. With the help of pointers, C++ creates dynamic variables. C++ provides two operators, `new` and `delete`, to create and destroy dynamic variables, respectively. When a program requires a new variable, the operator `new` is used. When a program no longer needs a dynamic variable, the operator `delete` is used.

In C++, `new` and `delete` are reserved words.

Operator `new`

The operator `new` has two forms: one to allocate a single variable, and another to allocate an array of variables. The syntax to use the operator `new` is as follows:

```
new dataType;            //to allocate a single variable
new dataType[intExp];    //to allocate an array of variables
```

where `intExp` is any expression evaluating to a positive integer.

The operator **new** allocates memory (a variable) of the designated type and returns a pointer to it—that is, the address of this allocated memory. Moreover, the allocated memory is uninitialized.

Consider the following declaration:

```
int *p;
char *q;
int x;
```


The statement

```
p = &x;
```

stores the address of **x** in **p**. However, no new memory is allocated. On the other hand, consider the following statement:

```
p = new int;
```

This statement creates a variable during program execution somewhere in memory, and stores the address of the allocated memory in **p**. The allocated memory is accessed via pointer dereferencing—namely, ***p**. Similarly, the statement

```
q = new char[16];
```

creates an array of 16 components of type **char** and stores the base address of the array in **q**.

Because a dynamic variable is unnamed, it cannot be accessed directly. It is accessed indirectly by the pointer returned by **new**. The following statements illustrate this concept:

```
int *p;   //p is a pointer of type int

p = new int;  //allocates memory of type int and stores the address
              //of the allocated memory in p
*p = 28;      //stores 28 in the allocated memory
```

NOTE

The operator **new** allocates memory space of a specific type and returns the (starting) address of the allocated memory space. However, if the operator **new** is unable to allocate the required memory space (for example, there is not enough memory space), the program might terminate with an error message.

Operator delete

Suppose you have the following declaration:

```
int *p;
```

This statement declares **p** to be a pointer variable of type **int**. Next, consider the following statements:

```
p = new int;      //Line 1
*p = 54;          //Line 2
p = new int;      //Line 3
*p = 73;          //Line 4
```

Let us see the effect of these statements. The statement in Line 1 allocates memory space of type **int** and stores the address of the allocated memory space into **p**. Suppose that the address of allocated memory space is **1500**. Then, the value of **p** after the execution of this statement is **1500**. (See Figure 3-2.)

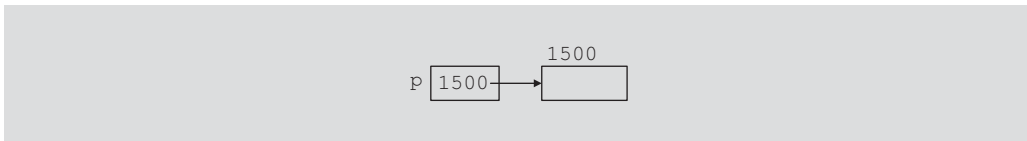


FIGURE 3-2 p after the execution of `p = new int;`

In Figure 3-2, the number **1500** on top of the box indicates the address of the memory space. The statement in Line 2 stores **54** into the memory space to which **p** points, which is **1500**. In other words, after execution of the statement in Line 2, the value stored into memory space at location **1500** is **54**. (See Figure 3-3.)

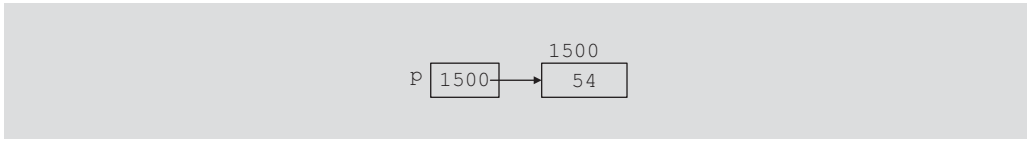


FIGURE 3-3 p and *p after the execution of `*p = 54;`

Next, the statement in Line 3 executes, which allocates a memory space of type **int** and stores the address of the allocated memory space into **p**. Suppose the address of this allocated memory space is **1800**. It follows that the value of **p** is now **1800**. (See Figure 3-4.)

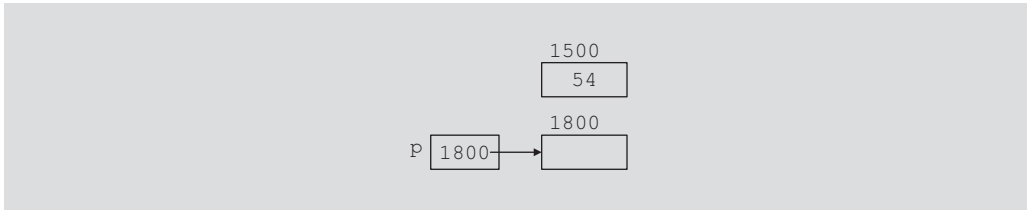


FIGURE 3-4 p after the execution of `p = new int;`

The statement in Line 4 stores **73** into the memory space to which **p** points, which is **1800**. In other words, after execution of the statement in Line 4, the value stored into the memory space at location **1800** is **73**. (See Figure 3-5.)

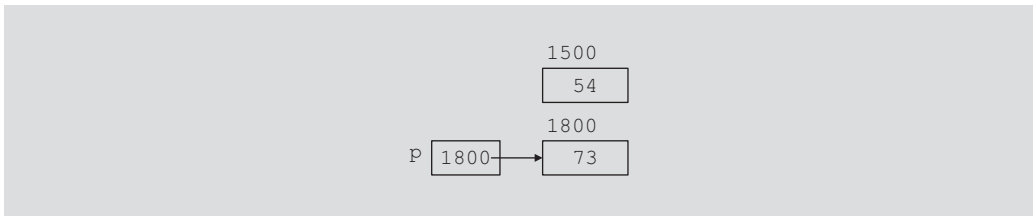


FIGURE 3-5 p after the execution of *p = 73;

Now the obvious question is, what happened to the memory space 1500, to which p was pointing, before the execution of the statement in Line 3? After execution of the statement in Line 3, p points to the new memory space at location 1800. The previous memory space at location 1500 is now inaccessible. In addition, the memory space 1500 remains marked as allocated. In other words, it cannot be reallocated. This is called **memory leak**. That is, there is an unused memory space that cannot be allocated.

Imagine what would happen if you execute statements such as Line 1 a few thousand times, or a few million times. There will be a good amount of memory leak. The program might then run out of memory space for data manipulation, and eventually result in an abnormal termination of the program.

The question at hand is how to *avoid* memory leak. When a dynamic variable is no longer needed, it can be destroyed; that is, its memory can be deallocated. The C++ operator **delete** is used to destroy dynamic variables, so that its memory space can be allocated again when needed. The syntax to use the operator **delete** has the following two forms:

```
delete pointerVariable; //to deallocate a single dynamic variable
delete [] pointerVariable; //to deallocate a dynamic array
```

Thus, given the declarations of the previous section, the statements

```
delete p;
delete str;
```

deallocate the memory spaces to which the pointers **p** and **str** point.

Suppose **p** is a pointer variable, as declared previously. Note that an expression such as

```
delete p;
```

only marks as deallocated the memory spaces to which these pointer variables point. Depending on a particular system, after these statements execute, these pointer variables might still contain the addresses of the deallocated memory spaces. In this case, we say that these pointers are **dangling**. Therefore, if you later access the memory spaces via these pointers without properly initializing them, depending on a particular system, either the program will access a wrong memory space, which might result in corrupting data, or the program will terminate with an error message. One way to avoid this pitfall is to set these pointers to **NULL** after the **delete** operation.

The program in the following example illustrates how to allocate dynamic memory and how to manipulate data into that dynamic memory.

EXAMPLE 3-3

```
//***********************************************************
// Author: D.S. Malik
//
// This program illustrates how to allocate dynamic memory
// using a pointer variable and how to manipulate data into
// that memory location.
//***********************************************************

#include <iostream>                                    //Line 1

using namespace std;                                   //Line 2

int main()                                             //Line 3
{                                                      //Line 4
    int *p;                                            //Line 5
    int *q;                                            //Line 6

    p = new int;                                       //Line 7
    *p = 34;                                           //Line 8
    cout << "Line 9: p = " << p
         << ", *p = " << *p << endl;                   //Line 9

    q = p;                                             //Line 10
    cout << "Line 11: q = " << q
         << ", *q = " << *q << endl;                   //Line 11

    *q = 45;                                           //Line 12
    cout << "Line 13: p = " << p
         << ", *p = " << *p << endl;                   //Line 13
    cout << "Line 14: q = " << q
         << ", *q = " << *q << endl;                   //Line 14

    p = new int;                                       //Line 15
    *p = 18;                                           //Line 16
    cout << "Line 17: p = " << p
         << ", *p = " << *p << endl;                   //Line 17
    cout << "Line 18: q = " << q
         << ", *q = " << *q << endl;                   //Line 18

    delete q;                                          //Line 19
    q = NULL;                                          //Line 20
    q = new int;                                       //Line 21
    *q = 62;                                           //Line 22
    cout << "Line 23: p = " << p
         << ", *p = " << *p << endl;                   //Line 23
    cout << "Line 24: q = " << q
         << ", *q = " << *q << endl;                   //Line 24

    return 0;                                          //Line 25
}                                                      //Line 26
```

Sample Run:

```
Line 9: p = 00355620, *p = 34
Line 11: q = 00355620, *q = 34
Line 13: p = 00355620, *p = 45
Line 14: q = 00355620, *q = 45
Line 17: p = 003556C8, *p = 18
Line 18: q = 00355620, *q = 45
Line 23: p = 003556C8, *p = 18
Line 24: q = 00355620, *q = 62
```


The statements in Lines 5 and 6 declare **p** and **q** to be pointer variables of type **int**. The statement in Line 7 allocates memory of type **int** and stores the address of the allocated memory into **p**. (See Figure 3-6.)

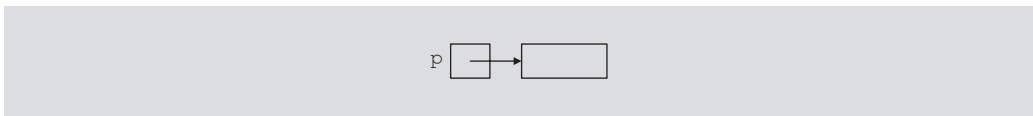


FIGURE 3-6 Pointer p and the memory space to which it points

The box indicates the allocated memory (in this case, of type **int**), and **p** together with the arrow indicates that **p** points to the allocated memory. The statement in Line 8 stores **34** into the memory location to which **p** points. (See Figure 3-7.)

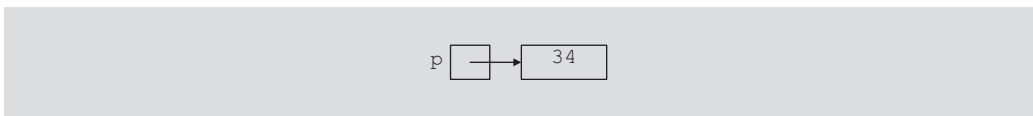


FIGURE 3-7 Pointer p and the value of the memory location to which p points

The statement in Line 9 outputs the value of **p** and ***p**. (Note that the values of **p** and **q** shown in the sample run are machine dependent. When you execute this program, you are likely to get different values of **p** and **q**.)

The statement in Line 10 copies the value of **p** into **q**. (See Figure 3-8.)

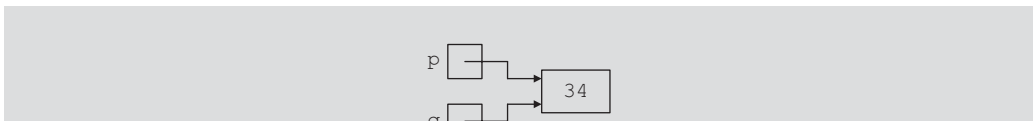


FIGURE 3-8 Pointers p and q and the memory space to which they point after the execution of the statement in Line 10

After the execution of the statement in Line 10, **p** and **q** both point to the same memory location. So any changes made into that memory location by **q** immediately change the value of ***p**. The statement in Line 11 outputs the value of **q** and ***q**. The statement in Line 12 stores **45** into the memory location to which **q** points. (See Figure 3-9.)

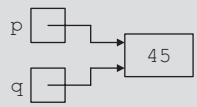


FIGURE 3-9 Pointers p and q and the memory space to which they point after the execution of the statement in Line 12

The statements in Lines 13 and 14 output the values of **p**, ***p**, **q**, and ***q**.

The statement in Line 15 allocates memory space of type **int** and stores the address of that memory into **p**. (See Figure 3-10.)

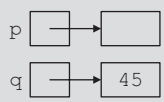


FIGURE 3-10 Pointers p and q and the memory space to which they point after the execution of the statement in Line 15

The statement in Line 16 stores **18** into the memory location to which **p** points. (See Figure 3-11.)

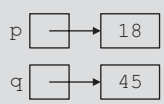


FIGURE 3-11 Pointers p and q and the memory space to which they point after the execution of the statement in Line 16

The statements in Lines 17 and 18 output the values of **p**, ***p**, **q**, and ***q**.

The statement in Line 19 deallocates the memory space to which **q** points and the statement in Line 20 sets the value of **q** to **NULL**. After the execution of the statement in Line 20, **q** does not point to any memory location. (See Figure 3-12.)

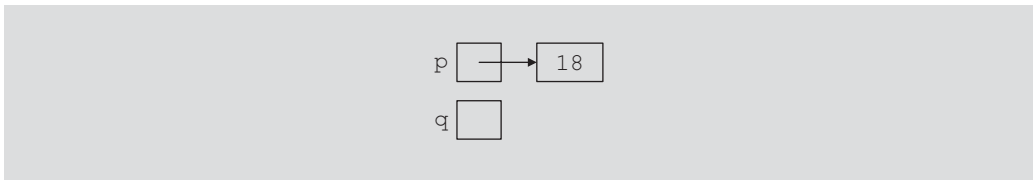


FIGURE 3-12 Pointers p and q and the memory space to which they point after the execution of the statement in Line 20

The statement in Line 21 allocates a memory space of type **int** and stores the address of that memory space into **q**. The statement in Line 22 stores **62** in the memory space to which **q** points. (See Figure 3-13.)

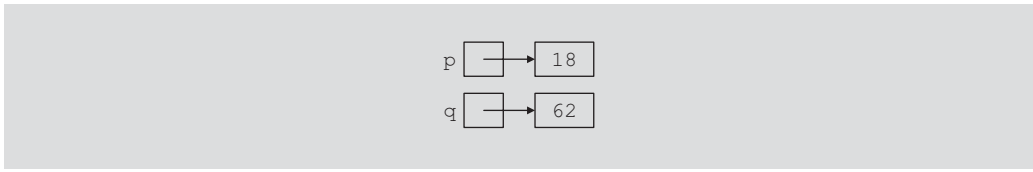


FIGURE 3-13 Pointers p and q and the memory space to which they point after the execution of the statement in Line 22

The statements in Lines 23 and 24 output the values of **p**, ***p**, **q**, and ***q**.

In the preceding program, omit statements in Lines 19 and 20, rerun the program, and note how the last output statements change.

Operations on Pointer Variables

The operations that are allowed on pointer variables are the assignment and relational operations and some limited arithmetic operations. The value of one pointer variable can be assigned to another pointer variable of the same type. Two pointer variables of the same type can be compared for equality, and so on. Integer values can be added and subtracted from a pointer variable. The value of one pointer variable can be subtracted from another pointer variable.

For example, suppose that we have the following statements:

```
int *p, *q;
```

The statement

```
p = q;
```

copies the value of **q** into **p**. After this statement executes, both **p** and **q** point to the same memory location. Any changes made to ***p** automatically change the value of ***q**, and vice versa.

The expression

```
p == q
```

evaluates to **true** if p and q have the same value—that is, if they point to the same memory location. Similarly, the expression

```
p != q
```

evaluates to true if p and q point to different memory locations.

The arithmetic operations that are allowed differ from the arithmetic operations on numbers. First, let us use the following statements to explain the increment and decrement operations on pointer variables:

```
int *p;
double *q;
char *chPtr;
```

Suppose that the size of the memory allocated for an int variable is 4 bytes, a double variable is 8 bytes, and a char variable is 1 byte.

The statement

```
p++;
```
or `p = p + 1;`

increments the value of p by 4 bytes because p is a pointer of type **int**. Similarly, the statements

```
q++;
chPtr++;
```

increment the value of q by 8 bytes and the value of chPtr by 1 byte, respectively.

The increment operator increments the value of a pointer variable by the size of the memory to which it is pointing. Similarly, the decrement operator decrements the value of a pointer variable by the size of the memory to which it is pointing.

Moreover, the statement

```
p = p + 2;
```

increments the value of p by 8 bytes.

Thus, when an integer is added to a pointer variable, the value of the pointer variable is incremented by the integer times the size of the memory to which the pointer is pointing. Similarly, when an integer is subtracted from a pointer variable, the value of the pointer variable is decremented by the integer times the size of the memory to which the pointer is pointing.

NOTE

Pointer arithmetic can be quite dangerous. Using pointer arithmetic, the program can accidentally access the memory locations of other variables and change their content without warning. The programmer is then left to try to find out what went wrong. If a pointer variable tries to access either the memory spaces of other variables or an illegal memory space, some systems might terminate the program with an appropriate error message. Always exercise extra care when doing pointer arithmetic.

Dynamic Arrays

The arrays used earlier are called static arrays because their size was fixed at compile time. One of the limitations of a static array is that every time you execute the program, the size of the array is fixed, so it might not be possible to use the same array to process different data sets of the same type. One way to handle this limitation is to declare an array that is large enough to process a variety of data sets. However, if the array is big and the data set is small, such a declaration would result in memory waste. On the other hand, it would be helpful if, during program execution, you could prompt the user to enter the size of the array and then create an array of the appropriate size. This approach is especially helpful if you cannot even guess the array size. In this section, you learn how to create arrays during program execution and how to process such arrays.

An array created during the execution of a program is called a **dynamic array**. To create a dynamic array, we use the second form of the **new** operator.

The statement

```
int *p;
```

declares p to be a pointer variable of type **int**. The statement

```
p = new int[10];
```

allocates 10 contiguous memory locations, each of type **int**, and stores the address of the first memory location into p. In other words, the operator **new** creates an array of 10 components of type **int**, it returns the base address of the array, and the assignment operator stores the base address of the array into p. Thus, the statement

```
*p = 25;
```

stores 25 into the first memory location, and the statements

```
p++;       //p points to the next array component
*p = 35;
```

store 35 into the second memory location. Thus, by using the increment and decrement operations, you can access the components of the array. Of course, after performing a few increment operations, it is possible to lose track of the first array component. C++ allows us to use array notation to access these memory locations. For example, the statements

```
p[0] = 25;
p[1] = 35;
```

store 25 and 35 into the first and second array components, respectively. That is, p[0] refers to the first array component, p[1] refers to the second array component, and so on. In general, p[i] refers to the (i + 1)th array component. After the preceding statements execute, p still points to the first array component.

The following **for** loop initializes each array component to 0:

```
for (int j = 0; j < 10; j++)
    p[j] = 0;
```

When the array notation is used to process the array to which `p` points, `p` stays fixed at the first memory location. Note that `p` is a dynamic array, created during program execution.

EXAMPLE 3-4

The following program segment illustrates how to obtain a user's response to get the array size and create a dynamic array during program execution. Consider the following statements:

```
int *intList;                       //Line 1
int arraySize;                      //Line 2

cout << "Enter array size: ";       //Line 3
cin >> arraySize;                   //Line 4
cout << endl;                       //Line 5

intList = new int[arraySize];       //Line 6
```

The statement in Line 1 declares `intList` to be a pointer of type `int`, and the statement in Line 2 declares `arraySize` to be an `int` variable. The statement in Line 3 prompts the user to enter the size of the array, and the statement in Line 4 inputs the array size into the variable `arraySize`. The statement in Line 6 creates an array of the size specified by `arraySize`, and the base address of the array is stored in `intList`. From this point on, you can treat `intList` just like any other array. For example, you can use the array notation to process the elements of `intList` and pass `intList` as a parameter to the function.

Array Name: A Constant Pointer

The statement

```
int list[5];
```

declares `list` to be an array of five components. Recall that `list` itself is a variable and the value stored in `list` is the base address of the array—that is, it is the address of the first array component. Suppose the address of the first array component is `1000`. Figure 3-14 shows `list` and the array `list`.

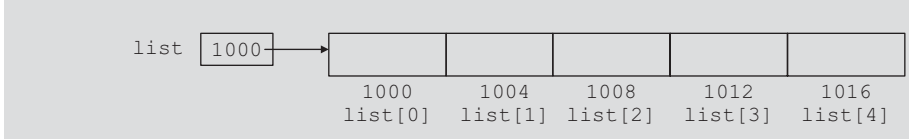


FIGURE 3-14 `list` and array `list`

Because the value of `list`, which is `1000`, is a memory address, `list` is a pointer variable. However, the value stored in `list`, which is `1000`, *cannot be altered during program execution.* That is, the value of `list` is *constant.* Therefore, the increment and decrement operations cannot be applied to `list`. In fact, any attempt to use the increment or decrement operations on `list` results in a compile-time error.

Notice that here we are *only* saying that the value of `list` cannot be changed. However, the data in the array `list` can be manipulated as usual. For example, the statement `list[0] = 25;` stores `25` into the first array component. Similarly, the statement `list[2] = 78;` stores `78` into the third component of `list`. (See Figure 3-15.)

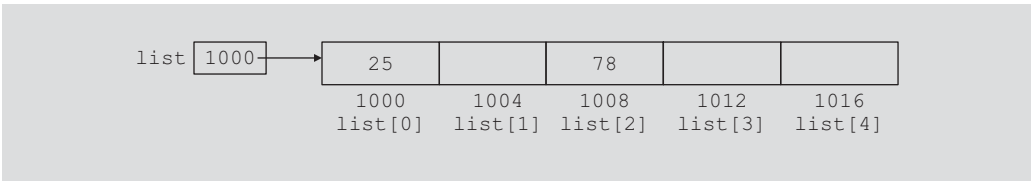


FIGURE 3-15 Array `list` after the execution of the statements `list[0] = 25;` and `list[2] = 78;`

If `p` is a pointer variable of type `int`, then the statement

```
p = list;
```

copies the value of `list`, which is `1000`, the base address of the array, into `p`. We are allowed to perform increment and decrement operations on `p`.

An *array name* is a *constant pointer.*

Functions and Pointers

A pointer variable can be passed as a parameter to a function either by value or by reference. To declare a pointer as a value parameter in a function heading, you use the same mechanism as you use to declare a variable. To make a formal parameter be a reference parameter, you use `&` when you declare the formal parameter in the function heading. Therefore, to declare a formal parameter as a reference parameter, you must use `&`. Between the data type name and the identifier name, you must include `*` to make the identifier a pointer and `&` to make it a reference parameter. The obvious question is: In what order should `&` and `*` appear between the data type name and the identifier to declare a pointer as a reference parameter? In C++, to make a pointer a reference parameter in a function heading, `*` appears before the `&` between the data type name and the identifier. The following example illustrates this concept:

```
void example(int* &p, double *q)
{
    .
    .
    .
}
```

In this example, both `p` and `q` are pointers. The parameter `p` is a reference parameter; the parameter `q` is a value parameter.

Pointers and Function Return Values

In C++, the return type of a function can be a pointer. For example, the return type of the function

```
int* testExp(...)
{
    .
    .
    .
}
```

is a pointer of type `int`.

Dynamic Two-Dimensional Arrays

The beginning of this section discussed how to create dynamic one-dimensional arrays. You can also create dynamic multidimensional arrays. In this section, we discuss how to create dynamic two-dimensional arrays. Dynamic multidimensional arrays are created similarly.

There are various ways you can create dynamic two-dimensional arrays. One way is as follows. Consider the statement:

```
int *board[4];
```

This statement declares `board` to be an array of four pointers wherein each pointer is of type `int`. Because `board[0]`, `board[1]`, `board[2]`, and `board[3]` are pointers, you can now use these pointers to create the rows of `board`. Suppose that each row of board has six columns. Then the following `for` loop creates the rows of `board`.

```
for (int row = 0; row < 4; row++)
    board[row] = new int[6];
```

Note that the expression `new int[6]` creates an array of six components of type `int` and returns the base address of the array. The assignment statement then stores the returned address into `board[row]`. It follows that after the execution of the previous `for` loop, `board` is a two-dimensional array of 4 rows and 6 columns.

In the previous `for` loop, if you replace the number 6 with the number 10, the loop will create a two-dimensional array of 4 rows and 10 columns. In other words, the number of columns of `board` can be specified during execution. However, the way `board` is declared, the number of rows is fixed. So in reality, `board` is not a true dynamic two-dimensional array.

Next, consider the following statement:

```
int **board;
```

This statement declares `board` to be a pointer to a pointer. In other words, `board` and `*board` are pointers. Now `board` can store the address of a pointer or an array of pointers of type `int`, and `*board` can store the address of an `int` memory space or an array of `int` values.

Suppose that you want `board` to be an array of 10 rows and 15 columns. To accomplish this, first we create an array of 10 pointers of type `int` and assign the address of that array to `board`. The following statement accomplishes this:

```
board = new int* [10];
```

Next we create the columns of `board`. The following `for` loop accomplishes this:

```
for (int row = 0; row < 10; row++)
    board[row] = new int[15];
```


To access the components of `board`, you can use the array subscripting notation. For example, see the next example. Note that the number of rows and the number of columns of `board` can be specified during program execution. The program in Example 3-5 further explains how to create two-dimensional arrays.

EXAMPLE 3-5

```
//***********************************************************
// Author: D.S. Malik
//
// This program illustrates how to use two-dimensional dynamic
// arrays.
//***********************************************************

#include <iostream>                                          //Line 1
#include <iomanip>                                           //Line 2

using namespace std;                                         //Line 3

void fill(int **p, int rowSize, int columnSize);             //Line 4
void print(int **p, int rowSize, int columnSize);            //Line 5

int main()                                                   //Line 6
{                                                            //Line 7
    int **board;                                             //Line 8

    int rows;                                                //Line 9
    int columns;                                             //Line 10

    cout << "Line 11: Enter the number of rows "
         <<"and columns: ";                                  //Line 11
    cin >> rows >> columns;                                  //Line 12
    cout << endl;                                            //Line 13

        //Create the rows of board
    board = new int* [rows];                                 //Line 14

        //Create the columns of board
    for (int row = 0; row < rows; row++)                     //Line 15
        board[row] = new int[columns];                       //Line 16
```

```
       //Insert elements into board
    fill(board, rows, columns);                          //Line 17

    cout << "Line 18: Board:" << endl;                   //Line 18

       //Output the elements of board
    print(board, rows, columns);                         //Line 19

    return 0;                                            //Line 20
}                                                        //Line 21

void fill(int **p, int rowSize, int columnSize)
{
    for (int row = 0; row < rowSize; row++)
    {
        cout << "Enter " << columnSize << " number(s) for row "
             << "number " << row << ": ";
        for (int col = 0; col < columnSize; col++)
            cin >> p[row][col];
        cout << endl;
    }
}

void print(int **p, int rowSize, int columnSize)
{
    for (int row = 0; row < rowSize; row++)
    {
         for (int col = 0; col < columnSize; col++)
            cout << setw(5) << p[row][col];
         cout << endl;
    }
}
```

Sample Run: In this sample run, the user input is shaded.

```
Line 11: Enter the number of rows and columns: 3 4

Enter 4 number(s) for row number 0: 1 2 3 4

Enter 4 number(s) for row number 1: 5 6 7 8

Enter 4 number(s) for row number 2: 9 10 11 12

Line 18: Board:
    1    2    3    4
    5    6    7    8
    9   10   11   12
```

The preceding program contains the functions `fill` and `print`. The function `fill` prompts the user to enter the elements of a two-dimensional array of type `int`. The function `print` outputs the elements of a two-dimensional array of type `int`.

For the most part, the preceding output is self-explanatory. Let us look at the statements in the function `main`. The statement in Line 8 declares `board` to be a pointer to a pointer of type `int`. The statements in Lines 9 and 10 declare `int` variables `rows` and `columns`. The statement in Line 11 prompts the user to input the number of rows and number of columns. The statement in Line 12 stores the number of rows in the variable `rows` and the number of columns in the variable `columns`. The statement in Line 14 creates the rows of `board` and the `for` loop in Lines 15 and 16 creates the columns of `board`. The statement in Line 17 used the function `fill` to fill the array `board` and the statement in Line 19 uses the function `print` to output the elements of `board`.

Shallow Vs. Deep Copy and Pointers

In an earlier section, we discussed pointer arithmetic and explained that if we are not careful, one pointer might access the data of another (completely unrelated) pointer. This event might result in unsuspected or erroneous results. Here, we discuss another peculiarity of pointers. To facilitate the discussion, we will use diagrams to show pointers and their related memory.

Suppose that you have the following declarations:

```
int *first;
int *second;
```

Further suppose that `first` points to an `int` array, as shown in Figure 3-16.

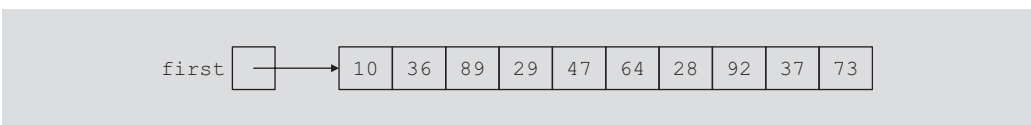


FIGURE 3-16 Pointer `first` and its array

Next, consider the following statement:

```
second = first;          //Line A
```

This statement copies the value of `first` into `second`. After this statement executes, both `first` and `second` point to the same array, as shown in Figure 3-17.

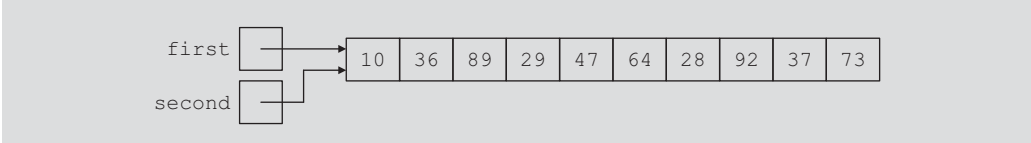


FIGURE 3-17 `first` and `second` after the statement `second = first;` executes

The statement `first[4] = 10;` not only changes the value of `first[4]`, it also changes the value of `second[4]` because they point to the same array.

Let us execute the following statement:

```
delete [] second;
```

After this statement executes, the array to which `second` points is deleted. This action results in Figure 3-18.

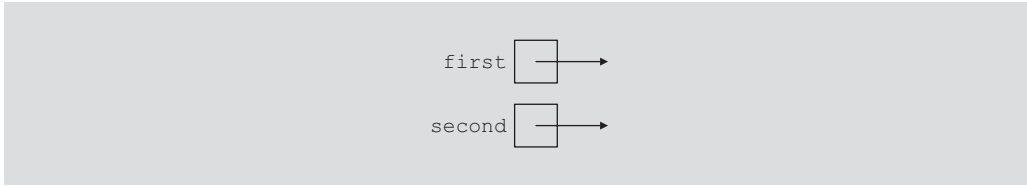


FIGURE 3-18 `first` and `second` after the statement `delete [] second;` executes

Because `first` and `second` pointed to the same array, after the statement

```
delete [] second;
```

executes, `first` becomes invalid, that is, `first` (as well as `second`) are now dangling pointers. Therefore, if the program later tries to access the memory to which `first` pointed, either the program will access the wrong memory or it will terminate in an error. This case is an example of a shallow copy. More formally, in a **shallow copy**, two or more pointers of the same type point to the same memory; that is, they point to the same data.

On the other hand, suppose that instead of the earlier statement, `second = first;` (in Line A), we have the following statements:

```
second = new int[10];

for (int j = 0; j < 10; j++)
    second[j] = first[j];
```

The first statement creates an array of 10 components of type `int`, and the base address of the array is stored in `second`. The second statement copies the array to which `first` points into the array to which `second` points. (See Figure 3-19.)

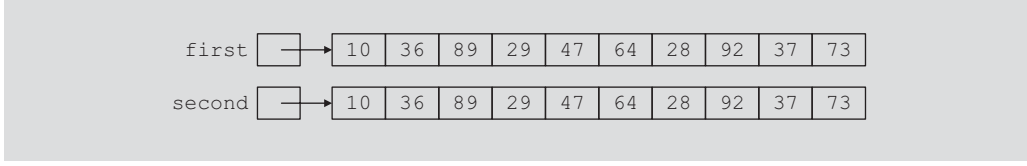


FIGURE 3-19 `first` and `second` both pointing to their own data

Both `first` and `second` now point to their own data. If `second` deletes its memory, there is no effect on `first`. This case is an example of a deep copy. More formally, in a **deep copy**, two or more pointers have their own data.

From the preceding discussion, it follows that you must know when to use a shallow copy and when to use a deep copy.

Classes and Pointers: Some Peculiarities

Because a class can have pointer member variables, this section discusses some peculiarities of such classes. To facilitate the discussion, we use the following class:

```
class pointerDataClass
{
public:
    .
    .
    .

private:
    int x;
    int lenP;
    int *p;
};
```

Also consider the following statements. (See Figure 3-20.)

```
pointerDataClass objectOne;
pointerDataClass objectTwo;
```

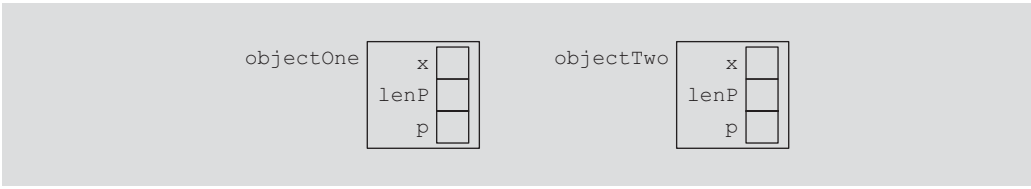


FIGURE 3-20 Objects `objectOne` and `objectTwo`

Destructor

The object `objectOne` has a pointer member variable `p`. Suppose that during program execution the pointer `p` creates a dynamic array. When `objectOne` goes out of scope, all the member variables of `objectOne` are destroyed. However, `p` created a dynamic array, and dynamic memory must be deallocated using the operator `delete`. Thus, if the pointer `p` does not use the `delete` operator to deallocate the dynamic array, the memory space of the dynamic array would stay marked as allocated, even though it cannot be

accessed. How do we ensure that when `p` is destroyed, the dynamic memory created by `p` is also destroyed? Suppose that `objectOne` is as shown in Figure 3-21.

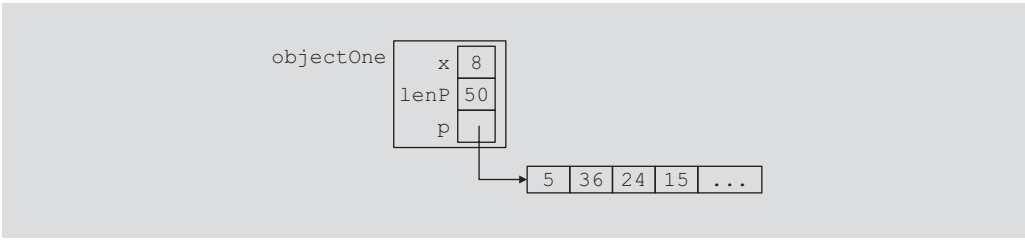


FIGURE 3-21 Object `objectOne` and its data

Recall that if a class has a destructor, the destructor automatically executes whenever a class object goes out of scope (see Chapter 1). Therefore, we can put the necessary code in the destructor to ensure that when `objectOne` goes out of scope, the memory created by the pointer `p` is deallocated. For example, the definition of the destructor for the `class pointerDataClass` is as follows:

```
pointerDataClass::~pointerDataClass()
{
    delete [] p;
}
```

Of course, you must include the destructor as a member of the class in its definition. Let us extend the definition of the `class pointerDataClass` by including the destructor. Moreover, the remainder of this section assumes that the definition of the destructor is as given previously—that is, the destructor deallocates the memory space pointed to by `p`.

```
class pointerDataClass
{
public:
    ~pointerDataClass();
      .
      .
      .

private:
    int x;
    int lenP;
    int *p;
};
```

NOTE For the destructor to work properly, the pointer `p` must have a valid value. If `p` is not properly initialized (that is, if the value of `p` is garbage) and the destructor executes, either the program terminates with an error message or the destructor deallocates an unrelated memory space. For this reason, you should exercise extra caution while working with pointers.

Assignment Operator

This section describes the limitations of the built-in assignment operators for classes with pointer member variables. Suppose that `objectOne` and `objectTwo` are as shown in Figure 3-22.

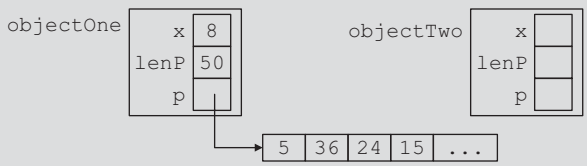


FIGURE 3-22 Objects `objectOne` and `objectTwo`

Recall that one of the built-in operations on classes is the assignment operator. For example, the statement:

```
objectTwo = objectOne;
```

copies the member variables of `objectOne` into `objectTwo`. That is, the value of `objectOne.x` is copied into `objectTwo.x`, and the value of `objectOne.p` is copied into `objectTwo.p`. Because `p` is a pointer, this memberwise copying of the data would lead to a shallow copying of the data. That is, both `objectTwo.p` and `objectOne.p` would point to the same memory space, as shown in Figure 3-23.

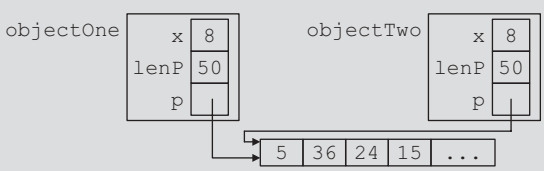


FIGURE 3-23 Objects `objectOne` and `objectTwo` after the statement `objectTwo = objectOne;` executes

Now, if `objectTwo.p` deallocates the memory space to which it points, `objectOne.p` would become invalid. This situation could very well happen, if the `class pointerDataClass` has a destructor that deallocates the memory space pointed to by `p` when an object of type `pointerDataClass` goes out of scope. It suggests that there must be a way to avoid this pitfall. To avoid this shallow copying of data for classes with a pointer member variable, C++ allows the programmer to extend the definition of the assignment operator. This process is called overloading the assignment operator. In the next section, we explain how to accomplish this task by using operator overloading. Once the assignment operator is properly overloaded, both `objectOne` and `objectTwo` have their own data, as shown in Figure 3-24.

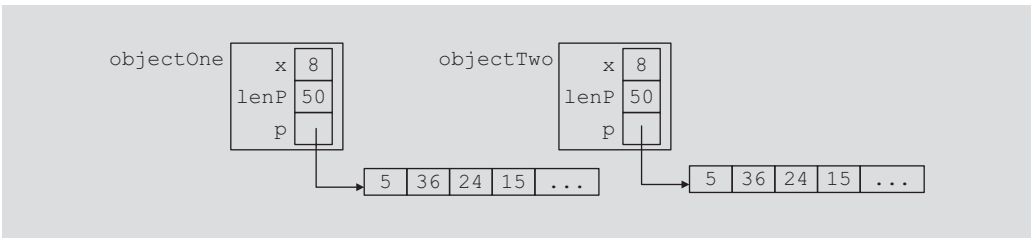


FIGURE 3-24 Objects `objectOne` and `objectTwo`

OVERLOADING THE ASSIGNMENT OPERATOR

Next we describe how to overload the assignment operator.

General Syntax to Overload the Assignment Operator = for a Class

Function Prototype (to be included in the definition of the class):

```
const className& operator=(const className&);
```

Function Definition:

```
const className& className::operator=(const className& rightObject)
{
         //local declaration, if any

      if (this != &rightObject)  //avoids self-assignment
      {
         //algorithm to copy rightObject into this object
      }

         //returns the object assigned
      return *this;
}
```

In the definition of the function `operator=`:

- There is only one formal parameter.
- The formal parameter is generally a `const` reference to a particular class.
- The return type of the function is a reference to a particular class.

Consider the statement

```
x = x;
```

Here, we are trying to copy the value of `x` into `x`; that is, this statement is a self-assignment. We must prevent such statements because they waste computer time.

The body of the function `operator=` does prevent such assignments. Let us see how.

Consider the `if` statement in the body of the operator function `operator=`:

```
if (this != &rightObject)  //avoids self-assignment
{
    //algorithm to copy rightObject into this object
}
```

Now the statement

```
x = x;
```

is compiled into the statement

```
x.operator=(x);
```

Because the function `operator=` is invoked by the object `x`, the pointer `this` in the body of the function `operator=` refers to the object `x`. Furthermore, because `x` is also a parameter to the function `operator=`, the formal parameter `rightObject` also refers to the object `x`. Therefore, in the expression

```
this != &rightObject
```

`this` means the address of `x`, and `&rightObject` also means the address of `x`. Thus, this expression will evaluate to `false` and, therefore, the body of the `if` statement will be skipped.

Notice that the return type of the function to overload the assignment operator is a reference. This is so that the statements such as `x = y = z`; can be executed, that is, the assignment operator can be used in a cascaded form.

In the section "Array-Based Lists," later in this chapter, we explicitly illustrate how to overload the assignment operator.

Copy Constructor

When declaring a class object, you can initialize it by using the value of an existing object of the same type. For example, consider the following statement:

```
pointerDataClass objectThree(objectOne);
```

The object `objectThree` is being declared and is also being initialized by using the value of `objectOne`. That is, the values of the member variables of `objectOne` are copied into the corresponding member variables of `objectThree`. This initialization is called the default memberwise initialization. The default memberwise initialization is due to the constructor, called the **copy constructor** (provided by the compiler). Just as in the case of the assignment operator, because the `class pointerDataClass` has pointer member variables, this default initialization would lead to a shallow copying of the data, as shown in Figure 3-25. (Assume that `objectOne` is given as before.)

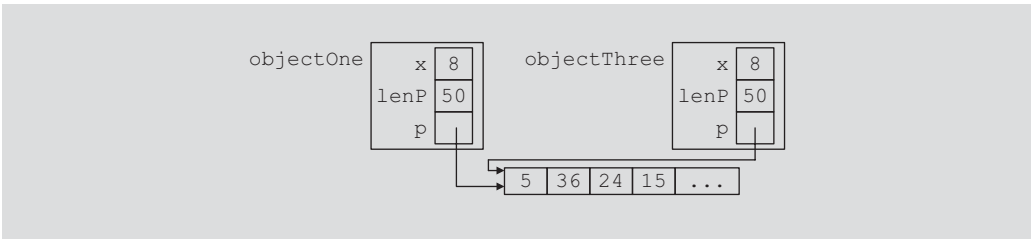


FIGURE 3-25 Objects `objectOne` and `objectThree`

Before describing how to overcome this deficiency, let us describe one more situation that could also lead to a shallow copying of the data. The solution to both these problems is the same.

Recall that, as parameters to a function, class objects can be passed either by reference or by value. Remember that the `class pointerDataClass` has the destructor, which deallocates the memory space pointed to by `p`. Suppose that `objectOne` is as shown in Figure 3-26.

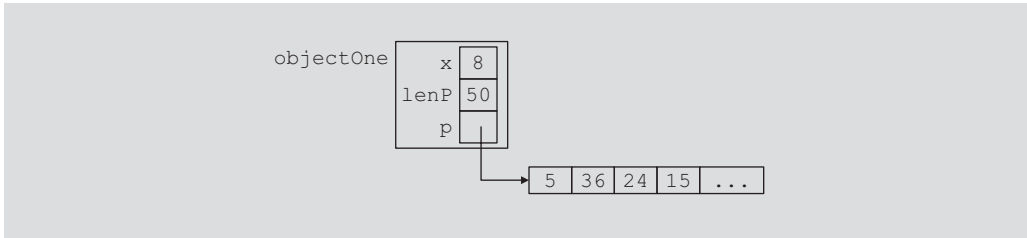


FIGURE 3-26 Object `objectOne`

Let us consider the following function prototype:

```
void destroyList(pointerDataClass paramObject);
```

The function `pointerDataClass` has a formal value parameter, `paramObject`. Now consider the following statement:

```
destroyList(objectOne);
```

In this statement, `objectOne` is passed as a parameter to the function `destroyList`. Because `paramObject` is a value parameter, the copy constructor copies the member variables of `objectOne` into the corresponding member variables of `paramObject`. Just as in the previous case, `paramObject.p` and `objectOne.p` would point to the same memory space, as shown in Figure 3-27.

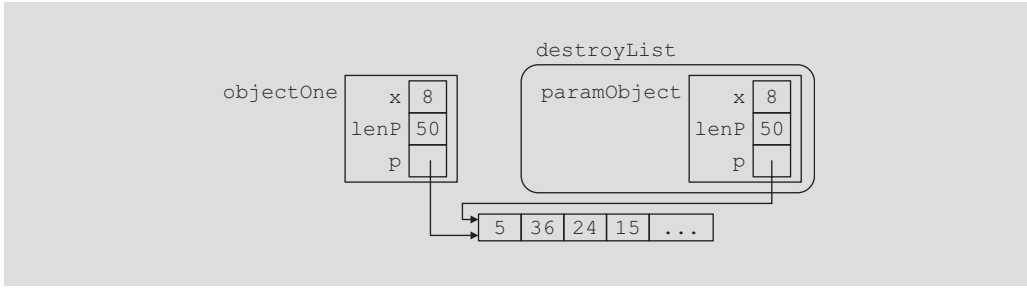


FIGURE 3-27 Pointer member variables of objects `objectOne` and `paramObject` pointing to the same array

Because `objectOne` is passed by value, the member variables of `paramObject` should have their own copy of the data. In particular, `paramObject.p` should have its own memory space. How do we ensure that this is, in fact, the case?

If a class has pointer member variables:

- During object declaration, the initialization of one object using the value of another object leads to a shallow copying of the data, if the default memberwise copying of data is allowed.
- If, as a parameter, an object is passed by value and the default memberwise copying of data is allowed, it leads to a shallow copying of the data.

In both cases, to force each object to have its own copy of the data, we must override the definition of the copy constructor provided by the compiler; that is, we must provide our own definition of the copy constructor. This is usually done by putting a statement that includes the copy constructor in the definition of the class, and then writing the definition of the copy constructor. Then, whenever the copy constructor needs to be executed, the system would execute the definition provided by us, not the one provided by the compiler. Therefore, for the `class pointerDataClass`, we can overcome this shallow copying problem by including the copy constructor in the `class pointerDataClass`.

The copy constructor automatically executes in three situations (the first two are described previously):

- When an object is declared and initialized by using the value of another object
- When, as a parameter, an object is passed by value
- When the return value of a function is an object

Therefore, once the copy constructor is properly defined for the `class pointerDataClass`, both `objectOne.p` and `objectThree.p` will have their own copies of the data. Similarly, `objectOne.p` and `paramObject.p` will have their own copies of the data, as shown in Figure 3-28.

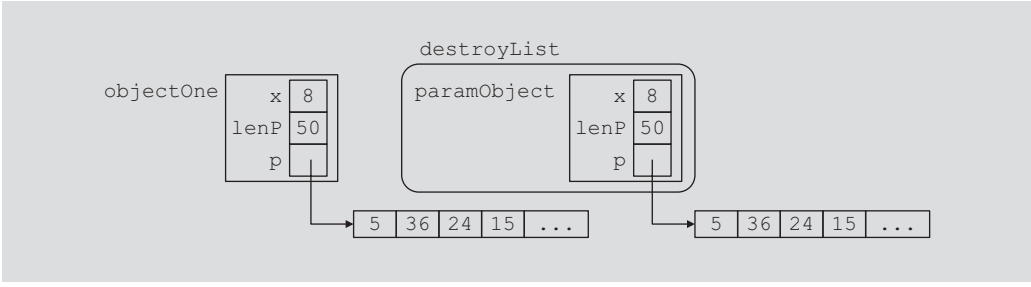


FIGURE 3-28 Pointer member variables of objects `objectOne` and `paramObject` with their own data

When the function `destroyList` exits, the formal parameter `paramObject` goes out of scope, and the destructor for the object `paramObject` deallocates the memory space pointed to by `paramObject.p`. However, this deallocation has no effect on `objectOne`.

The general syntax to include the copy constructor in the definition of a class is as follows:

```
className(const className& otherObject);
```

Notice that the formal parameter of the copy constructor is a constant reference parameter.

In the section, "Array-Based Lists," we explicitly illustrate how to include the copy constructor in a class and how it works.

For classes with pointer member variables, three things are normally done:

1. Include the destructor in the class.
2. Overload the assignment operator for the class.
3. Include the copy constructor.

Inheritance, Pointers, and Virtual Functions

Recall that, as a parameter, a class object can be passed either by value or by reference. Earlier chapters also said that the types of the actual and formal parameters must match. However, in the case of classes, C++ *allows the user to pass an object of a derived class to a formal parameter of the base class type.*

First, let us discuss the case when the formal parameter is either a reference parameter or a pointer. To be specific, let us consider the following classes:

```
class baseClass
{
public:
    void print();
    baseClass(int u = 0);

private:
    int x;
};

class derivedClass: public baseClass
{
public:
    void print();
    derivedClass(int u = 0, int v = 0);

private:
    int a;
};
```

The `class baseClass` has three members. The `class derivedClass` is derived from the `class baseClass`, and has three members of its own. Both classes have a member function `print`. Suppose that the definitions of the member functions of both classes are as follows:

```
void baseClass::print()
{
    cout << "In baseClass x = " << x << endl;
}

baseClass::baseClass(int u)
{
    x = u;
}

void derivedClass::print()
{
    cout << "In derivedClass ***: ";
    baseClass::print();
    cout << "In derivedClass a = " << a << endl;
}

derivedClass::derivedClass(int u, int v)
               : baseClass(u)
{
    a = v;
}
```

Consider the following function in a user program (client code):

```
void callPrint(baseClass& p)
{
    p.print();
}
```

The function `callPrint` has a formal reference parameter `p` of type `baseClass`. You can call the function `callPrint` by using an object of either type `baseClass` or type `derivedClass` as a parameter. Moreover, the body of the function `callPrint` calls the member function `print`. Consider the following function `main`:

```
int main()                                      //Line 1
{                                               //Line 2
    baseClass one(5);                           //Line 3
    derivedClass two(3, 15);                    //Line 4

    one.print();                                //Line 5
    two.print();                                //Line 6

    cout << "*** Calling the function "
         << "callPrint  ***" << endl;           //Line 7

    callPrint(one);                             //Line 8
    callPrint(two);                             //Line 9

    return 0;                                   //Line 10
}                                               //Line 11
```

Sample Run:

```
In baseClass x = 5
In derivedClass ***: In baseClass x = 3
In derivedClass a = 15
*** Calling the function callPrint  ***
In baseClass x = 5
In baseClass x = 3
```

The statements in Lines 3 through 7 are quite straightforward. Let us look at the statements in Lines 8 and 9. The statement in Line 8 calls the function `callPrint` and passes the object `one` as the parameter; it generates the fifth line of the output. The statement in Line 9 also calls the function `callPrint`, but passes the object `two` as the parameter; it generates the sixth line of the output. The output generated by the statements in Lines 8 and 9 shows only the value of `x`, even though in these statements a different class object is passed as a parameter. (Because in Line 9 object `two` is passed as a parameter to the function `callPrint`, one would expect that the output generated by the statement in Line 9 should be similar to the output in the second and third lines of the output.) What actually occurred is that for both statements (Lines 8 and 9), the member function `print` of the `class baseClass` is executed. This is due to the fact that the binding of the member function `print`, in the body of the function `callPrint`, occurred at compile time. Because the formal parameter `p` of the function `callPrint` is of type `baseClass`, for the statement `p.print();`, the compiler associates the function `print` of the `class baseClass`. More specifically, in **compile-time binding**, the necessary code to call a specific function is generated by the compiler. (Compile-time binding is also known as **static binding**.)

For the statement in Line 9, the actual parameter is of type `derivedClass`. Thus, when the body of the function `callPrint` executes, logically the `print` function of object `two` should execute, which is not the case. So, during program execution, how does C++ correct this problem of making the call to the appropriate function? C++ corrects this problem by providing the mechanism of **virtual functions**. The binding of virtual functions occurs at program execution time, not at compile time. This kind of binding is called **run-time binding**. More formally, in run-time binding, the compiler does not generate the code to call a specific function. Instead, it generates enough information to enable the run-time system to generate the specific code for the appropriate function call. Run-time binding is also known as **dynamic binding**.

In C++, virtual functions are declared using the reserved word `virtual`. Let us redefine the previous classes using this feature:

```
class baseClass
{
public:
    virtual void print();       //virtual function
    baseClass(int u = 0);

private:
    int x;
};
```

```
class derivedClass: public baseClass
{
public:
    void print();
    derivedClass(int u = 0, int v = 0);

private:
    int a;
};
```


Note that we need to declare a `virtual` function only in the base `class`.

The definition of the member function `print` is the same as before. If we execute the previous program with these modifications, the output is as follows.

Sample Run:

```
In baseClass x = 5
In derivedClass ***: In baseClass x = 3
In derivedClass a = 15
*** Calling the function callPrint  ***
In baseClass x = 5
In derivedClass ***: In baseClass x = 3
In derivedClass a = 15
```

This output shows that for the statement in Line 9, the `print` function of `derivedClass` is executed (see the last two lines of the output).

The previous discussion also applies when a formal parameter is a pointer to a class, and a pointer of the derived class is passed as an actual parameter. To illustrate this feature, suppose we have the preceding classes. (We assume that the definition of the `class baseClass` is in the header file `baseClass.h`, and the definition of the `class derivedClass` is in the header file `derivedClass.h`.) Consider the following program:

```
//*****************************************************
// Author: D.S. Malik
//
// This program illustrates how virtual functions and
// pointer formal parameters work.
//*****************************************************

#include <iostream>                          //Line 1

#include "derivedClass.h"                    //Line 2

using namespace std;                         //Line 3

void callPrint(baseClass *p);                //Line 4

int main()                                   //Line 5
{                                            //Line 6
    baseClass *q;                            //Line 7
    derivedClass *r;                         //Line 8
```

```
    q = new baseClass(5);                          //Line 9
    r = new derivedClass(3, 15);                   //Line 10

    q->print();                                    //Line 11
    r->print();                                    //Line 12

    cout << "*** Calling the function "
         << "callPrint  ***" << endl;              //Line 13

    callPrint(q);                                  //Line 14
    callPrint(r);                                  //Line 15

    return 0;                                      //Line 16
}                                                  //Line 17

void callPrint(baseClass *p)
{
    p->print();
}
```

Sample Run:

```
In baseClass x = 5
In derivedClass ***: In baseClass x = 3
In derivedClass a = 15
*** Calling the function callPrint  ***
In baseClass x = 5
In derivedClass ***: In baseClass x = 3
In derivedClass a = 15
```

The preceding examples show that if a formal parameter, say `p` of a class type, is either a reference parameter or a pointer and `p` uses a virtual function of the base class, we can effectively pass a derived class object as an actual parameter to `p`.

However, if `p` is a *value parameter*, then this mechanism of passing a derived class object as an actual parameter to `p` does not work, even if `p` uses a virtual function. Recall that, if a formal parameter is a value parameter, the value of the actual parameter is copied into the formal parameter. Therefore, if a formal parameter is of a `class` type, the member variables of the actual object are copied into the corresponding member variables of the formal parameter.

Suppose that we have the classes defined above—that is, `baseClass` and `derivedClass`. Consider the following function definition:

```
void callPrint(baseClass p)  //p is a value parameter
{
    p.print();
}
```

Further suppose that we have the following declaration:

```
derivedClass two;
```

The object two has two member variables, x and a. The member variable x is inherited from the base class. Consider the following function call:

```
callPrint(two);
```

In this statement, because the formal parameter p is a value parameter, the member variables of two are copied into the member variables of p. However, because p is an object of type baseClass, it has only one member variable. Consequently, only the member variable x of two will be copied into the member variable x of p. Also, the statement:

```
p.print();
```

in the body of the function will result in executing the member function print of the class baseClass.

The output of the following program further illustrates this concept. (As before, we assume that the definition of the class baseClass is in the header file baseClass.h, and the definition of the class derivedClass is in the header file derivedClass.h.)

```
//*****************************************************
// Author: D.S. Malik
//
// This program illustrates how virtual functions and a
// pointer variable of base class as a formal parameter
// work.
//*****************************************************

#include <iostream>                              //Line 1

#include "derivedClass.h"                        //Line 2

using namespace std;                             //Line 3

void callPrint(baseClass p);                     //Line 4

int main()                                       //Line 5
{                                                //Line 6
    baseClass one(5);                            //Line 7
    derivedClass two(3, 15);                     //Line 8

    one.print();                                 //Line 9
    two.print();                                 //Line 10

    cout << "*** Calling the function "
         << "callPrint  ***" << endl;            //Line 11

    callPrint(one);                              //Line 12
    callPrint(two);                              //Line 13

    return 0;                                    //Line 14
}                                                //Line 15
```

```
void callPrint(baseClass p)  //p is a value parameter
{
    p.print();
}
```

Sample Run:

```
In baseClass x = 5
In derivedClass ***: In baseClass x = 3
In derivedClass a = 15
*** Calling the function callPrint  ***
In baseClass x = 5
In baseClass x = 3
```

Look closely at the output of the statements in Lines 12 and 13 (the last two lines of output). In Line 13, because the formal parameter `p` is a value parameter, the member variables of `two` are copied into the corresponding member variables of `p`. However, because `p` is an object of type `baseClass`, it has only one member variable. Consequently, only the member variable `x` of `two` is copied into the member variable `x` of `p`. Moreover, the statement `p.print();` in the function `callPrint` executes the function `print` of the `class baseClass`, not the `class derivedClass`. Therefore, the last line of the output shows only the value of x (the member variable of `two`).

NOTE An object of the base class type cannot be passed to a formal parameter of the derived class type.

Classes and Virtual Destructors

One thing recommended for classes with pointer member variables is that these classes should have the destructor. The destructor is automatically executed when the class object goes out of scope. Thus, if the object creates dynamic objects, the destructor can be designed to deallocate the storage for them. If a derived class object is passed to a formal parameter of the base class type, the destructor of the base class executes regardless of whether the derived class object is passed by reference or by value. Logically, however, the destructor of the derived class should be executed when the derived class object goes out of scope.

To correct this problem, the destructor of the base class must be virtual. The **virtual destructor** of a base class automatically makes the destructor of a derived class virtual. When a derived class object is passed to a formal parameter of the base class type, then when the object goes out of scope, the destructor of the derived class executes. After executing the destructor of the derived class, the destructor of the base class executes. Therefore, when the derived class object is destroyed, the base class part (that is, the members inherited from the base class) of the derived class object is also destroyed.

If a base class contains virtual functions, make the destructor of the base class virtual.

Abstract Classes and Pure Virtual Functions

The preceding section discussed virtual functions. Other than enforcing run-time binding of functions, virtual functions also have another use, which is discussed in this section. Chapter 2 described the second principal of OOD—inheritance. Through inheritance we can derive new classes without designing them from scratch. The derived classes, in addition to inheriting the existing members of the base class, can add their own members and also redefine or override public and protected member functions of the base class. The base class can contain functions that you would want each derived class to implement. There are many scenarios when a class is desired to be served as a base class for a number of derived classes, however, the base class may contain certain functions that may not have meaningful definitions in the base class.

Let us consider the `class shape` given in Chapter 2. As noted in that chapter, from the `class shape` you can derive other `class`es such as `rectangle`, `circle`, `ellipse`, and so on. Some of the things common to every shape are its center, using the center to move a shape to a different location, and drawing the shape. Among others, we can include these in the `class shape`. For example, you could have the definition of the `class shape` similar to the following:

```
class shape
{
public:
    virtual void draw();
      //Function to draw the shape.

    virtual void move(double x, double y);
      //Function to move the shape at the position (x, y).
    .
    .
    .
};
```

Because the definitions of the functions `draw` and `move` are specific to a particular shape, each derived class can provide an appropriate definition of these functions. Note that we have made the functions `draw` and `move` `virtual` to enforce run-time binding of these functions.

The way the definition of the `class shape` is written when you write the definition of the functions of the `class shape`, you must also write the definitions of the functions `draw` and `move`. However, at this point there is no shape to draw or move. Therefore, these function bodies have no code. One way to handle this is to make the body of these functions empty. This solution would work, but it has another drawback. Once we write the definitions of the functions of the `class shape`, then we could create an object of this class. Because there is no shape to work with, we would like to prevent the user from creating objects of the `class shape`. It follows that we would like to do the following two things—to not include the definitions of the functions `draw` and `move`, and to prevent the user from creating objects of the `class shape`.

Because we do not want to include the definitions of the functions `draw` and `move` of the `class shape`, we must convert these functions to **pure virtual functions**. In this case, the prototypes of these functions are:

```
virtual void draw() = 0;
virtual void move(double x, double y) = 0;
```

Note the expression `= 0` before the semicolon. Once you make these functions pure `virtual` functions in the `class shape`, you no longer need to provide the definitions of these functions for the `class shape`.

Once a class contains one or more pure virtual functions, then that class is called an **abstract class**. Thus, the abstract definition of the `class shape` is similar to the following:

```
class shape
{
public:
    virtual void draw() = 0;
      //Function to draw the shape. Note that this is a
      //pure virtual function.

    virtual void move(double x, double y) = 0;
      //Function to move the shape at the position (x, y).
      //Note that this is a pure virtual function.
    .
    .
    .
};
```

Because an abstract class is not a complete class, as it (or its implementation file) does not contain the definitions of certain functions, you cannot create objects of that class.

Now suppose that we derive the `class rectangle` from the `class shape`. To make `rectangle` a nonabstract class, so that we can create objects of this class, the class (or its implementation file) must provide the definitions of the pure `virtual` functions of its base class, which is the `class shape`.

Note that in addition to the pure virtual functions, an abstract class can contain instance variables, constructors, and functions that are not pure virtual. However, the abstract class must provide the definitions of constructor and functions that are not pure virtual.

Array-Based Lists

Everyone is familiar with the term *list*. You might have a list consisting of employee data, student data, sales data, or a list of rental properties. One thing common to all lists is that all the elements of a list are of the same type. More formally, we can define a list as follows:

List: A collection of elements of the same type.

The **length** of a list is the number of elements in the list.

Following are some of the operations performed on a list:

1. Create the list. The list is initialized to an empty state.
2. Determine whether the list is empty.
3. Determine whether the list is full.
4. Find the size of the list.
5. Destroy, or clear, the list.
6. Determine whether an item is the same as a given list element.
7. Insert an item in the list at the specified location.
8. Remove an item from the list at the specified location.
9. Replace an item at the specified location with another item.
10. Retrieve an item from the list from the specified location.
11. Search the list for a given item.

Before discussing how to implement these operations, we must first decide how to store the list in the computer's memory. Because all the elements of a list are of the same type, an effective and convenient way to process a list is to store it in an array. Initially, the size of the array holding the list elements is usually larger than the number of elements in the list so that, at a later stage, the list can grow. Thus, we must know how full the array is; that is, we must keep track of the number of list elements stored in the array. C++ allows the programmer to create dynamic arrays. Therefore, we leave it for the user to specify the size of the array. The size of the array can be specified when a list object is declared. It follows that, to maintain and process the list in an array, we need the following three variables:

- The array holding the list elements
- A variable to store the length of the list (that is, the number of list elements currently in the array)
- A variable to store the size of the array (that is, the maximum number of elements that can be stored in the array)

Suppose that the variable `length` indicates the number of elements in the list and that `maxSize` indicates the maximum number of elements that can be stored in the list. Then `length` and `maxSize` are nonnegative integers and, therefore, we can declare them to be of type `int`. What about the type of the array, that is, the data type of the array elements? If we have a list of numbers, the array elements could be of type `int` or `double`. If we have a list of names, the array elements are of type `string`. Similarly, if we have a list of students, the array elements are of type `studentType` (a data type you can define). As you can see, there are various types of lists.

A list of sales data or a list of students' data is empty if its length is zero. To insert an item at the end of a list of any type would require you to add the element after the current last element and then increment the length by one. Similarly, it can be seen that, for the most part, the algorithms to implement operations on a list of names, on a list of sales data, or on a list of students' data are the same. We do not want to spend time and efforts to

develop separate code for each type of list we encounter. Instead, we want to develop a generic code that can be used to implement any type of list in a program. In other words, while designing the algorithms, we do not want to be concerned whether we are processing a list of numbers, a list of names, or a list of students' data. However, while illustrating a particular algorithm, we will consider a specific type of list. To develop generic algorithms to implement list operations, we make use of class templates.

Now that we know the operations to be performed on a list and how to store the list into the computer's memory, next we define the class implementing the list as an abstract data type (ADT). The following `class`, `arrayListType`, defines the list as an ADT:

```
//*************************************************************
// Author: D.S. Malik
//
// This class specifies the members to implement the basic
// properties of array-based lists.
//*************************************************************

template <class elemType>
class arrayListType
{
public:
    const arrayListType<elemType>& operator=
                        (const arrayListType<elemType>&);
      //Overloads the assignment operator
    bool isEmpty() const;
      //Function to determine whether the list is empty
      //Postcondition: Returns true if the list is empty;
      //    otherwise, returns false.
    bool isFull() const;
      //Function to determine whether the list is full.
      //Postcondition: Returns true if the list is full;
      //    otherwise, returns false.
    int listSize() const;
      //Function to determine the number of elements in the list
      //Postcondition: Returns the value of length.
    int maxListSize() const;
      //Function to determine the size of the list.
      //Postcondition: Returns the value of maxSize.
    void print() const;
      //Function to output the elements of the list
      //Postcondition: Elements of the list are output on the
      //   standard output device.
    bool isItemAtEqual(int location, const elemType& item) const;
      //Function to determine whether the item is the same
      //as the item in the list at the position specified by
      //Postcondition: Returns true if list[location]
      //    is the same as the item; otherwise,
      //    returns false.
   void insertAt(int location, const elemType& insertItem);
      //Function to insert an item in the list at the
      //position specified by location. The item to be inserted
      //is passed as a parameter to the function.
```

```
    //Postcondition: Starting at location, the elements of the
    //    list are shifted down, list[location] = insertItem;,
    //    and length++;. If the list is full or location is
    //    out of range, an appropriate message is displayed.
  void insertEnd(const elemType& insertItem);
    //Function to insert an item at the end of the list.
    //The parameter insertItem specifies the item to be inserted.
    //Postcondition: list[length] = insertItem; and length++;
    //    If the list is full, an appropriate message is
    //    displayed.
  void removeAt(int location);
    //Function to remove the item from the list at the
    //position specified by location
    //Postcondition: The list element at list[location] is removed
    //    and length is decremented by 1. If location is out of
    //    range, an appropriate message is displayed.
  void retrieveAt(int location, elemType& retItem) const;
    //Function to retrieve the element from the list at the
    //position specified by location.
    //Postcondition: retItem = list[location]
    //    If location is out of range, an appropriate message is
    //    displayed.
  void replaceAt(int location, const elemType& repItem);
    //Function to replace the elements in the list at the
    //position specified by location. The item to be replaced
    //is specified by the parameter repItem.
    //Postcondition: list[location] = repItem
    //    If location is out of range, an appropriate message is
    //    displayed.
  void clearList();
    //Function to remove all the elements from the list.
    //After this operation, the size of the list is zero.
    //Postcondition: length = 0;
  int seqSearch(const elemType& item) const;
    //Function to search the list for a given item.
    //Postcondition: If the item is found, returns the location
    //    in the array where the item is found; otherwise,
    //    returns -1.
  void insert(const elemType& insertItem);
    //Function to insert the item specified by the parameter
    //insertItem at the end of the list. However, first the
    //list is searched to see whether the item to be inserted
    //is already in the list.
    //Postcondition: list[length] = insertItem and length++
    //     If the item is already in the list or the list
    //     is full, an appropriate message is displayed.
  void remove(const elemType& removeItem);
    //Function to remove an item from the list. The parameter
    //removeItem specifies the item to be removed.
    //Postcondition: If removeItem is found in the list,
    //      it is removed from the list and length is
    //      decremented by one.
```

```
    arrayListType(int size = 100);
      //constructor
      //Creates an array of the size specified by the
      //parameter size. The default array size is 100.
      //Postcondition: The list points to the array, length = 0,
      //    and maxSize = size

    arrayListType(const arrayListType<elemType>& otherList);
      //copy constructor

    ~arrayListType();
      //destructor
      //Deallocates the memory occupied by the array.

protected:
    elemType *list;  //array to hold the list elements
    int length;      //to store the length of the list
    int maxSize;     //to store the maximum size of the list
};
```

Figure 3-29 shows the UML class diagram of the `class arrayListType`.

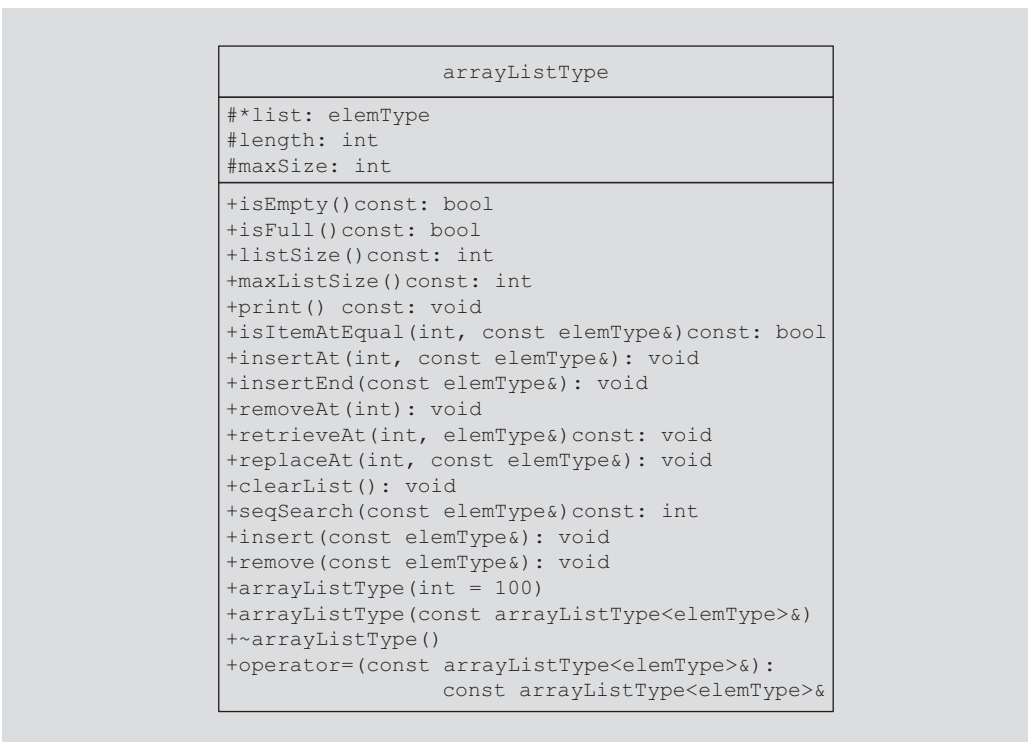


FIGURE 3-29 UML class diagram of the `class arrayListType`

Notice that the data members of the **class** **arrayListType** are declared as **protected**. This is because we want to derive classes from this class to implement special lists such as an ordered list. Next, we write the definitions of these functions.

The list is empty if **length** is **zero**; it is full if **length** is equal to **maxSize**. Therefore, the definitions of the functions **isEmpty** and **isFull** are as follows:

```
template <class elemType>
bool arrayListType<elemType>::isEmpty() const
{
    return (length == 0);
}

template <class elemType>
bool arrayListType<elemType>::isFull() const
{
    return (length == maxSize);
}
```

The data member **length** of the class stores the number of elements currently in the list. Similarly, because the size of the array holding the list elements is stored in the data member **maxSize**, **maxSize** specifies the maximum size of the list. Therefore, the definitions of the functions **listSize** and **maxListSize** are as follows:

```
template <class elemType>
int arrayListType<elemType>::listSize() const
{
    return length;
}

template <class elemType>
int arrayListType<elemType>::maxListSize() const
{
    return maxSize;
}
```

Each of the functions **isEmpty**, **isFull**, **listSize**, and **maxListSize** contain only one statement, which is either a comparison statement or a statement returning a value. It follows that each of these functions is of $O(1)$.

The member function **print** outputs the elements of the list. We assume that the output is sent to the standard output device.

```
template <class elemType>
void arrayListType<elemType>::print() const
{
    for (int i = 0; i < length; i++)
        cout << list[i] << " ";

    cout << endl;
}
```

The function **print** uses a loop to output the elements of the list. The number of times the **for** loop executes depends on the number of elements of the list. If the list has 100

elements, the **for** loop executes 100 times. In general, suppose that the number of elements in the list is *n*. Then the function `print` is of *O*(*n*).

The definition of the function `isItemAtEqual` is straightforward.

```
template <class elemType>
bool arrayListType<elemType>::isItemAtEqual
                            (int location, const elemType& item) const
{
    return(list[location] == item);
}
```

The body of function `isItemAtEqual` has only one statement, which is a comparison statement. It is easy to see that this function is of *O*(1).

The function `insertAt` inserts an item at a specific location in the list. The item to be inserted and the insert location in the array are passed as parameters to this function. To insert the item somewhere in the middle of the list, we must first make room for the new item. That is, we need to move certain elements right one array slot. Suppose that the data member `list` of an `arrayListType` object is as shown in Figure 3-30. (Note that this figure does not show the data members `length` and `maxSize`.)

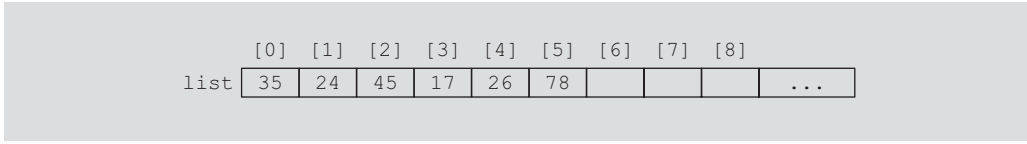


FIGURE 3-30 Array `list`

The number of elements currently in the list is `6`, so `length` is `6`. Thus, after inserting a new element, the `length` of the list is `7`. If the item is to be inserted at, say location `6`, we can easily accomplish this by copying the item into `list[6]`. On the other hand, if the item is to be inserted at, say location `3`, we first need to move elements `list[3]`, `list[4]`, and `list[5]` one array slot right to make room for the new item. Thus, we must first copy `list[5]` into `list[6]`, `list[4]` into `list[5]`, and `list[3]` into `list[4]`, in this order. Then we can copy the new item into `list[3]`.

Of course, special cases such as trying to insert in a full list must be handled separately. Other member functions can handle some of these cases.

The definition of the function `insertAt` is as follows:

```
template <class elemType>
void arrayListType<elemType>::insertAt
                    (int location, const elemType& insertItem)
{
    if (location < 0 || location >= maxSize)
        cerr << "The position of the item to be inserted "
             << "is out of range" << endl;
```

```
    else
        if (length >= maxSize)  //list is full
            cerr << "Cannot insert in a full list" << endl;
        else
        {
            for (int i = length; i > location; i--)
                 list[i] = list[i - 1];   //move the elements down

            list[location] = insertItem;  //insert the item at the
                                          //specified position

            length++;     //increment the length
        }
} //end insertAt
```


The function `insertAt` uses a **`for`** loop to shift the elements of the list. The number of times the **`for`** loop executes depends on where in the list the item is to be inserted. If the item is to be inserted at the first position, all the elements of the list are shifted. It can be easily shown that this function is of $O(n)$.

The function `insertEnd` can be implemented by using the function `insertAt`. However, the function `insertEnd` does not require the shifting of elements. Therefore, we give its definition directly.

```
template <class elemType>
void arrayListType<elemType>::insertEnd(const elemType& insertItem)
{
    if (length >= maxSize)  //the list is full
        cerr << "Cannot insert in a full list" << endl;
    else
    {
         list[length] = insertItem;   //insert the item at the end
         length++;   //increment the length
    }
} //end insertEnd
```

The number of statements and, hence, the number of operations executed in the body of the function `insertEnd` are fixed. Therefore, this function is of $O(1)$.

The function `removeAt` is the opposite of the function `insertAt`. The function `removeAt` removes an item from a specific location in the list. The location of the item to be removed is passed as a parameter to this function. After removing the item from the list, the length of the list is reduced by `1`. If the item to be removed is somewhere in the middle of the list, after removing the item we must move certain elements left one array slot because we cannot leave holes in the portion of the array containing the list. Suppose that the data member `list` of an `arrayListType` object is as shown in Figure 3-31. (Note that this figure does not show the data members `length` and `maxSize`.)

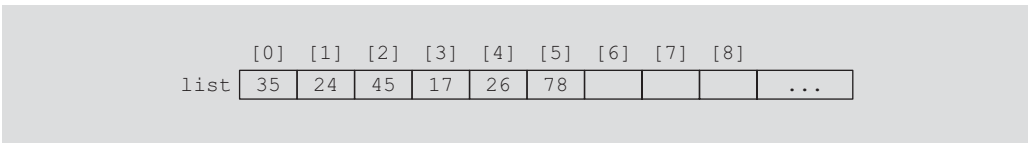


FIGURE 3-31 Array `list`

The number of elements currently in the list is `6`, so `length` is `6`. Thus, after removing an element, the `length` of the list is `5`. Suppose that the item to be removed is at, say location `3`. Clearly, we must move `list[4]` into `list[3]` and `list[5]` into `list[4]`, in this order.

The definition of the function `removeAt` is as follows:

```
template <class elemType>
void arrayListType<elemType>::removeAt(int location)
{
    if (location < 0 || location >= length)
        cerr << "The location of the item to be removed "
             << "is out of range" << endl;
    else
    {
        for (int i = location; i < length - 1; i++)
            list[i] = list[i+1];

        length--;
    }
} //end removeAt
```

Similar to the function `insertAt`, it is easily seen that the function `removeAt` is of $O(n)$.

The definition of the function `retrieveAt` is straightforward. The index of the item to be retrieved, and the location where to retrieve the item, are passed as parameters to this function. Similarly, the definition of the function `replaceAt` is straightforward. The definitions of these functions are as follows:

```
template <class elemType>
void arrayListType<elemType>::retrieveAt
                              (int location, elemType& retItem) const
{
    if (location < 0 || location >= length)
        cerr << "The location of the item to be retrieved is "
             << "out of range." << endl;
    else
        retItem = list[location];
} //end retrieveAt


template <class elemType>
void arrayListType<elemType>::replaceAt
                            (int location, const elemType& repItem)
```

```
{
    if (location < 0 || location >= length)
        cerr << "The location of the item to be replaced is "
             << "out of range." << endl;
    else
        list[location] = repItem;

} //end replaceAt
```


The function `clearList` removes the elements from the list, leaving it empty. Because the data member `length` indicates the number of elements in the list, the elements are removed by simply setting `length` to `0`. Therefore, the definition of this function is as follows:

```
template <class elemType>
void arrayListType<elemType>::clearList()
{
    length = 0;
} //end clearList
```

We now discuss the definition of the constructor and the destructor. The constructor creates an array of the size specified by the user, and initializes the `length` of the list to `0` and the `maxSize` to the size of the array specified by the user. The size of the array is passed as a parameter to the constructor. The default array size is 100. The destructor deallocates the memory occupied by the array holding the list elements. The definition of the constructor and the destructor are as follows:

```
template <class elemType>
arrayListType<elemType>::arrayListType(int size)
{
    if (size < 0)
    {
        cerr << "The array size must be positive. Creating "
             << "an array of size 100. " << endl;

        maxSize = 100;
    }
    else
        maxSize = size;

    length = 0;

    list = new elemType[maxSize];
    assert(list != NULL);
}

template <class elemType>
arrayListType<elemType>::~arrayListType()
{
    delete [] list;
}
```

As before, it is easy to see that each of the functions `retrieveAt`, `replaceAt`, `clearList`, as well as the constructor and destructor is of $O(1)$.

Copy Constructor

Recall that the copy constructor is called when an object is passed as a (value) parameter to a function, and when an object is declared and initialized using the value of another object of the same type. It copies the data members of the actual object into the corresponding data members of the formal parameter and the object being created. Its definition is as follows:

```
template <class elemType>
arrayListType<elemType>::arrayListType
                    (const arrayListType<elemType>& otherList)
{
    maxSize = otherList.maxSize;
    length = otherList.length;
    list = new elemType[maxSize]; //create the array
    assert(list != NULL);         //terminate if unable to allocate
                                  //memory space

    for (int j = 0; j < length; j++)  //copy otherList
        list [j] = otherList.list[j];
} //end copy constructor
```

Overloading the Assignment Operator

Next, because we are overloading the assignment operator for the `class arrayListType`, we give the definition of the function template to overload the assignment operator.

```
template <class elemType>
const arrayListType<elemType>& arrayListType<elemType>::operator=
                      (const arrayListType<elemType>& otherList)
{
    if (this != &otherList)   //avoid self-assignment
    {
        delete [] list;
        maxSize = otherList.maxSize;
        length = otherList.length;

        list = new elemType[maxSize];  //create the array
        assert(list != NULL);   //if unable to allocate memory
                                //space, terminate the program
        for (int i = 0; i < length; i++)
            list[i] = otherList.list[i];
    }

    return *this;
}
```

Similar to the function `print`, it is easy to see that both the copy constructor and the function to overload the assignment operator are of $O(n)$.

Search

The search algorithm described next is called a **sequential** or **linear** search.

Consider the list of seven elements shown in Figure 3-32.

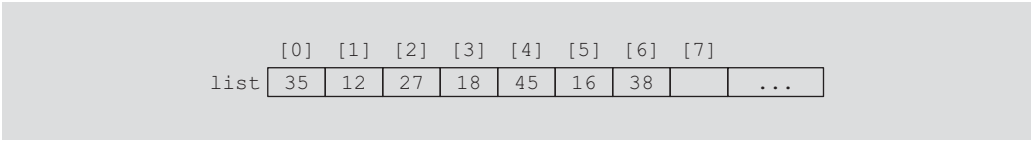


FIGURE 3-32 List of seven elements

Suppose that you want to determine whether `27` is in the list. The sequential search works as follows: First, you compare `27` with `list[0]`—that is, compare `27` with `35`. Because `list[0]` $\neq$ `27`, you then compare `27` with `list[1]` (that is, with `12`, the second item in the list). Because `list[1]` $\neq$ `27`, you compare `27` with the next element in the list—that is, compare `27` with `list[2]`. Because `list[2]` = `27`, the search stops. This is a successful search.

Let us now search for `10`. As before, the search starts with the first element in the list—that is, at `list[0]`. This time the search item, which is `10`, is compared with every item in the list. Eventually, no more data is left in the list to compare with the search item. This is an unsuccessful search.

It now follows that, as soon as you find an element in the list that is equal to the search item, you must stop the search and report "success." (In this case, you usually also tell the location in the list where the search item was found.) Otherwise, after the search item is compared with every element in the list, you must stop the search and report "failure."

Suppose that the name of the array containing the list elements is `list`. The following function performs a sequential search on a list:

```
template <class elemType>
int arrayListType<elemType>::seqSearch(const elemType& item) const
{
    int loc;
    bool found = false;

    for (loc = 0; loc < length; loc++)
        if (list[loc] == item)
        {
            found = true;
            break;
        }
```

```
    if (found)
        return loc;
    else
        return -1;
} //end seqSearch
```

Now that we know how to implement the (sequential) search algorithm, we can give the definitions of the functions `insert` and `remove`. Recall that the function `insert` inserts a new item at the end of the list if this item does not exist in the list and the list is not full. The function `remove` removes an item from the list if the list is not empty.

Chapter 9 explicitly shows that the function `seqSearch` is of $O(n)$.

Insert

The function `insert` inserts a new item in the list. Because duplicates are not allowed, this function first searches the list to determine whether the item to be inserted is already in the list. To determine whether the item to be inserted is already in the list, this function calls the member function `seqSearch`, described previously. If the item to be inserted is not in the list, the new item is inserted at the end of the list and the length of the list is increased by 1. Also, the item to be inserted is passed as a parameter to this function. The definition of this function is as follows:

```
template <class elemType>
void arrayListType<elemType>::insert(const elemType& insertItem)
{
    int loc;

    if (length == 0)   //list is empty
        list[length++] = insertItem;    //insert the item and
                                        //increment the length
    else if (length == maxSize)
        cerr << "Cannot insert in a full list." << endl;
    else
    {
        loc = seqSearch(insertItem);

        if (loc == -1)    //the item to be inserted
                          //does not exist in the list
            list[length++] = insertItem;
        else
            cerr << "the item to be inserted is already in "
                 << "the list. No duplicates are allowed." << endl;
    }
} //end insert
```

The function `insert` uses the function `seqSearch` to determine whether the `insertItem` is already in the list. Because the function `seqSearch` is of $O(n)$, it follows that the function `insert` is of $O(n)$.

Remove

The function `remove` deletes an item from the list. The item to be deleted is passed as a parameter to this function. To delete the item, the function calls the member function `seqSearch` to determine whether the item to be deleted is in the list. If the item to be deleted is found in the list, the item is removed from the list and the length of the list is decremented by 1. If the item to be removed is found in the list, the function `seqSearch` returns the `index` of the item in the list to be deleted. We can now use the `index` returned by the function `seqSearch`, and use the function `removeAt` to remove the item from the list. Therefore, the definition of the function `remove` is as follows:

```
template<class elemType>
void arrayListType<elemType>::remove(const elemType& removeItem)
{
    int loc;

    if (length == 0)
        cerr << "Cannot delete from an empty list." << endl;
    else
    {
        loc = seqSearch(removeItem);

        if (loc != -1)
            removeAt(loc);
        else
            cout << "The item to be deleted is not in the list."
                 << endl;
    }
} //end remove
```

The function `remove` uses the functions `seqSearch` and `removeAt` to remove an item from the list. Because each of these functions is of $O(n)$ and because they are called in sequence, it follows that the function `remove` is of $O(n)$.

Time Complexity of List Operations

The following table summarizes the time complexity of list operations.

TABLE 3-1 Time complexity of list operations

Function	Time-complexity
`isEmpty`	$O(1)$
`isFull`	$O(1)$
`listSize`	$O(1)$
`maxListSize`	$O(1)$

TABLE 3-1 Time complexity of list operations (continued)

Function	Time-complexity
print	$O(n)$
isItemAtEqual	$O(1)$
insertAt	$O(n)$
insertEnd	$O(1)$
removeAt	$O(n)$
retrieveAt	$O(1)$
replaceAt	$O(n)$
clearList	$O(1)$
constructor	$O(1)$
destructor	$O(1)$
copy constructor	$O(n)$
overloading the assignment operator	$O(n)$
seqSearch	$O(n)$
insert	$O(n)$
remove	$O(n)$

The following program tests the various operations on array-based lists.

```
//*****************************************************************
// Author: D.S. Malik
//
// This program illustrates how to use the class arrayListType.
//*****************************************************************

#include <iostream>                                          //Line 1

#include <string>                                            //Line 2
#include "arrayListType.h"                                   //Line 3
```

```
using namespace std;                                              //Line 4

int main()                                                        //Line 5
{                                                                 //Line 6
    arrayListType<int> intList(100);                              //Line 7
    arrayListType<string> stringList;                             //Line 8

    int number;                                                   //Line 9

    cout << "List 10: Enter 5 integers: ";                        //Line 10

    for (int counter = 0; counter < 5; counter++)                 //Line 11
    {                                                             //Line 12
        cin >> number;                                            //Line 13
        intList.insertAt(counter, number);                        //Line 14
    }                                                             //Line 15

    cout << endl;                                                 //Line 16
    cout << "List 19: The list you entered is: ";                 //Line 17
    intList.print();                                              //Line 18
    cout << endl;                                                 //Line 19

    cout << "Line 20: Enter the item to be deleted: ";            //Line 20
    cin >> number;                                                //Line 21
    intList.remove(number);                                       //Line 22
    cout << "Line 23: After removing " << number
         << ", the list is:" << endl;                             //Line 23
    intList.print();                                              //Line 24
    cout << endl;                                                 //Line 25

    string str;                                                   //Line 26

    cout << "Line 27: Enter 5 strings: ";                         //Line 27

    for (int counter = 0; counter < 5; counter++)                 //Line 28
    {                                                             //Line 29
        cin >> str;                                               //Line 30
        stringList.insertAt(counter, str);                        //Line 31
    }                                                             //Line 32

    cout << endl;                                                 //Line 33
    cout << "Line 34: The list you entered is: " << endl;         //Line 34
    stringList.print();                                           //Line 35
    cout << endl;                                                 //Line 36

    cout << "Line 37: Enter the string to be deleted: ";          //Line 37
    cin >> str;                                                   //Line 38
    stringList.remove(str);                                       //Line 39
    cout << "Line 40: After removing " << str
         << ", the list is:" << endl;                             //Line 40
```

```
    stringList.print();                                          //Line 41
    cout << endl;                                                //Line 42

    return 0;                                                    //Line 43
}                                                                //Line 44
```

Sample Run: In this sample run, the user input is shaded.

```
List 10: Enter 5 integers: 23 78 56 12 79

List 19: The list you entered is: 23 78 56 12 79

Line 20: Enter the item to be deleted: 56
Line 23: After removing 56, the list is:
23 78 12 79

Line 27: Enter 5 strings: hello sunny warm winter summer

Line 34: The list you entered is:
hello sunny warm winter summer

Line 37: Enter the string to be deleted: hello
Line 40: After removing hello, the list is:
sunny warm winter summer
```

The preceding program works as follows. The statement in Line 7 declares `intList` to be an object of type `arrayListType`. The data member `list` of `intList` is an array of 100 components and the component type is `int`. The statement in Line 8 declares `stringList` to be an object of type `arrayListType`. The data member `list` of `stringList` is an array of 100 components (the default size) and the component type is `string`. The statement in Line 10 prompts the user to enter five integers. The statement in Line 13 gets the next number from the input stream. The statement in Line 14 uses the member function `insertAt` of `intList` to store the number into `intList`. The statement in Line 18 uses the member function `print` of `intList` to output the elements of `intList`. The statement in Line 20 prompts the user to enter the number to be deleted from `intList`; the statement in Line 21 gets the number to be deleted from the input stream. The statement in Line 22 uses the member function `remove` of `intList` to remove the number from `intList`.

The statements in Lines 27 through 42 work the same way as the statements in Lines 10 through 25. These statements process a list of strings.

PROGRAMMING EXAMPLE: Polynomial Operations

You learned in a college algebra or calculus course that a polynomial, $p(x)$, in one variable, x, is an expression of the form:

$$p(x) = a_0 + a_1x + \ldots + a_{n-1}x^{n-1} + a_nx^n,$$

where a_i are real (or complex) numbers and n is a nonnegative integer. If $p(x) = a_0$, $p(x)$ is called a **constant** polynomial. If $p(x)$ is a nonzero constant polynomial, the degree of $p(x)$ is defined to be 0. Even though, in mathematics, the degree of the zero polynomial is undefined, for the purpose of this program, we consider the degree of such polynomials to be zero. If $p(x)$ is not constant and $a_n \neq 0$, n is called the degree of $p(x)$; that is, the degree of a nonconstant polynomial is defined to be the exponent of the highest power of x. (Note that the symbol $\neq$ means not equal to.)

The basic operations performed on polynomials are add, subtract, multiply, divide, and evaluate a polynomial at any given point. For example, suppose that

$$p(x) = 1 + 2x + 3x^2,$$

and

$$q(x) = 4 + x.$$

The degree of $p(x)$ is 2 and the degree of $q(x)$ is 1. Moreover,

$$p(2) = 1 + 2 \cdot 2 + 3 \cdot 2^2 = 17$$
$$p(x) + q(x) = 5 + 3x + 3x^2$$
$$p(x) - q(x) = -3 + x + 3x^2$$
$$p(x)^*q(x) = 4 + 9x + 14x^2 + 3x^3$$

The purpose of this programming example is to design and implement the `class polynomialType` to perform the various polynomial operations in a program.

To be specific, in this program, we will implement the following operations on polynomials:

1. Evaluate a polynomial at a given value.
2. Add polynomials.
3. Subtract polynomials.
4. Multiply polynomials.

Furthermore, we assume that the coefficients of polynomials are real numbers. You will be asked in Programming Exercise 8 to generalize it so that the coefficients can also be complex numbers.To store a polynomial, we use a dynamic array as follows:

Suppose $p(x)$ is a polynomial of degree $n \geq 0$. Let `list` be an array of size $n + 1$. The coefficient a_i of x^i is stored in `list[i]`. See Figure 3-33.

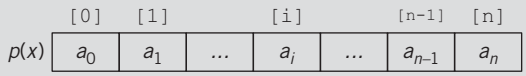


FIGURE 3-33 Polynomial $p(x)$

Figure 3-33 shows that if $p(x)$ is a polynomial of degree n, we need an array of size $n+1$ to store the coefficients of $p(x)$. Suppose that $p(x) = 1 + 8x - 3x^2 + 5x^4 + 7x^8$. Then the array storing the coefficient of $p(x)$ is given in Figure 3-34.

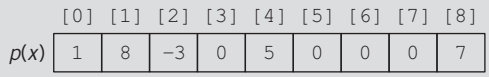


FIGURE 3-34 Polynomial $p(x)$ of degree 8 and its coefficients

Similarly, if $q(x) = -5x^2 + 16x^5$, the array storing the coefficient of $q(x)$ is given in Figure 3-35.

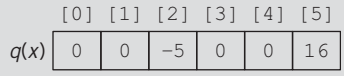


FIGURE 3-35 Polynomial $q(x)$ of degree 5 and its coefficients

Next, we define the operations +, –, and ⋆. Suppose that

$$p(x) = a_0 + a_1x + \ldots + a_{n-1}x^{n-1} + a_nx^n \text{ and}$$
$$q(x) = b_0 + b_1x + \ldots + b_{m-1}x^{m-1} + a_mx^m.$$

Let $t = max(n, m)$. Then

$$p(x) + q(x) = c_0 + c_1x + \ldots + c_{t-1}x^{t-1} + c_tx^t,$$

where for $i = 0, 1, 2, \ldots, t$

$$c_i = \begin{cases} a_i + b_i & if\ i \leq \min(n, m) \\ a_i & if\ i > m \\ b_i & if\ i > n \end{cases}$$

The difference, *p*(*x*) - *q*(*x*), of *p*(*x*) and *q*(*x*) can be defined similarly. It follows that the degree of the polynomials is $\leq$ *max*(*n*, *m*).

The product, *p*(*x*) * *q*(*x*), of *p*(*x*) and *q*(*x*) is defined as follows:

$p(x) * q(x) = d_0 + d_1x + \ldots + d_{n+m}x^{n+m}$,

The coefficient d_k, for $k = 0, 1, 2, \ldots, t$, is given by the formula

$d_k = a_0 * b_k + a_1 * b_{k-1} + \ldots + a_k * b_0$,

where if either a_i or b_i does not exist, it is assumed to be zero. For example,

$d_0 = a_0b_0$

$d_1 = a_0b_1 + a_1b_0$

...

$d_{n+m} = a_nb_m$

In Chapter 2, you learned how to overload various operators. This program overloads the operators `+`, `-`, and `*` to perform polynomial addition, subtraction, and multiplication. Moreover, we also overload the function call operator, `()`, to evaluate a polynomial at a given value. To simplify the input and output of polynomials, the operators `<<` and `>>` are also overloaded.

Because the coefficients of a polynomial are stored in a dynamic array, we use the `class arrayListType` to store and manipulate the coefficients of a polynomial. In fact, we derive the `class polynomialType` to implement polynomial operations from the `class arrayListType`, which requires us to implement only the operations needed to manipulate polynomials.

The following class defines polynomials as an ADT:

```
//***********************************************************
// Author: D.S. Malik
//
// This class specifies the members to implement the basic
// polynomial operations.
//***********************************************************

class polynomialType: public arrayListType<double>
{
    friend ostream& operator<<(ostream&, const polynomialType&);
      //Overloads the stream insertion operator
    friend istream& operator>>(istream&, polynomialType&);
      //Overloads the stream extraction operator
```

```
public:
    polynomialType operator+(const polynomialType&);
      //Overloads the operator +
    polynomialType operator-(const polynomialType&);
      //Overloads the operator -
    polynomialType operator*(const polynomialType&);
      //Overloads the operator *

    double operator() (double x);
      //Overloads the operator () to evaluate the polynomial at
      //a given point
      //Postcondition: The value of the polynomial at x is
      //    calculated and returned

    polynomialType(int size = 100);
      //constructor

    int min(int x, int y) const;
      //Function to return the smaller of x and y
    int max(int x, int y) const;
      //Function to return the larger of x and y
};
```

In Exercise 24 (at the end of this chapter), you are asked to draw the UML diagram of the `class polynomialType`.

If $p(x)$ is a polynomial of degree 3, we can create an object, say p, of type `polynomialType` and set the size of the array `list` to `4`. The following statement declares such an object p:

```
polynomialType p(4);
```

The degree of the polynomial is stored in the data member `length`, which is inherited from the `class arrayListType`.

Next we discuss the definitions of the functions.

The constructor sets the value of `length` to the size of the array and initializes the array `list` to `0`.

```
polynomialType::polynomialType(int size)
              : arrayListType<double>(size)
{
    length = size;

    for (int i = 0; i < size; i++)
        list[i] = 0;
}
```

The definition of the function to overload the operator `()` is quite straightforward and is given next.

```
double polynomialType::operator() (double x)
{
    double value = 0.0;

    for (int i = 0; i < length; i++)
    {
        if (list[i] != 0.0)
            value = value + list[i] * pow(x,i);
    }

    return value;
}
```

Suppose that $p(x)$ is a polynomial of degree n and $q(x)$ is a polynomial of degree m. If $n = m$, the operator + adds the corresponding coefficients of $p(x)$ and $q(x)$. If $n > m$, the first m coefficients of $p(x)$ are added with the corresponding coefficients of $q(x)$. The remaining coefficients of $p(x)$ are copied into the polynomial containing the sum of $p(x)$ and $q(x)$. Similarly, if $n < m$, the first n coefficients of $q(x)$ are added with the corresponding coefficients of $p(x)$. The remaining coefficients of $q(x)$ are copied into the polynomial containing the sum. The definition of the operator – is similar to the definition of the operator +. The definitions of these two operator functions are as follows:

```
polynomialType polynomialType::operator+
                              (const polynomialType& right)
{
    int size = max(length, right.length);

    polynomialType temp(size); //polynomial to store the sum

    for (int i = 0; i < min(length, right.length); i++)
        temp.list[i] = list[i] + right.list[i];

    if (size == length)
        for (int i = min(length, right.length); i < length; i++)
            temp.list[i] = list[i];
    else
        for (int i = min(length, right.length); i < right.length;
                                                i++)
            temp.list[i] = right.list[i];

    return temp;
}

polynomialType polynomialType::operator-
                               (const polynomialType& right)
{
    int size = max(length, right.length);

   polynomialType temp(size); //polynomial to store the difference
```

```
    for (int i = 0; i < min(length, right.length); i++)
        temp.list[i] = list[i] - right.list[i];

    if (size == length)
        for (int i = min(length, right.length); i < length; i++)
            temp.list[i] = list[i];
    else
        for (int i = min(length, right.length); i < right.length;
                                                i++)
            temp.list[i] = -right.list[i];

    return temp;
}
```

The definition of the function to overload the operator * to multiply two polynomials is left as an exercise for you. See Programming Exercise 6 at the end of this chapter. The definitions of the remaining functions of the `class polynomialType` are as follows:

```
int polynomialType::min(int x, int y) const
{
    if (x <= y)
        return x;
    else
        return y;
}

int polynomialType::max(int x, int y) const
{
    if (x >= y)
        return x;
    else
        return y;
}

ostream& operator<<(ostream& os, const polynomialType& p)
{
   int indexFirstNonzeroCoeff = 0;

   for (int i = 0; i < p.length; i++)  //determine the index of the
                                       //first nonzero coefficient
        if (p.list[i] != 0.0)
        {
            indexFirstNonzeroCoeff = i;
            break;
        }

    if (indexFirstNonzeroCoeff < p.length)
    {
        if (indexFirstNonzeroCoeff == 0)
            os << p.list[indexFirstNonzeroCoeff] << " ";
```

```
        else
            os << p.list[indexFirstNonzeroCoeff] << "x^"
               << indexFirstNonzeroCoeff << " ";

        for (int i = indexFirstNonzeroCoeff + 1; i < p.length; i++)
        {
            if (p.list[i] != 0.0)
                if (p.list[i] >= 0.0)
                    os << "+ " << p.list[i] << "x^" << i << " ";
                else
                    os << "- " << -p.list[i] << "x^" << i << " ";
        }
    }
    else
        os << "0";

    return os;
}

istream& operator>>(istream& is, polynomialType& p)
{
    cout << "The degree of this polynomial is: "
         << p.length - 1 << endl;

    for (int i = 0; i < p.length; i++)
    {
        cout << "Enter the coefficient of x^" << i << ": ";
        is >> p.list[i];
    }

    return is;
}
```

MAIN PROGRAM

```
//*****************************************************************
// Author: D.S. Malik
//
// This program illustrates how to use the class polynomialType.
//*****************************************************************

#include <iostream>                                          //Line 1

#include "polynomialType.h"                                  //Line 2

using namespace std;                                         //Line 3

int main()                                                   //Line 4
{                                                            //Line 5
    polynomialType p(8);                                     //Line 6
    polynomialType q(4);                                     //Line 7
    polynomialType t;                                        //Line 8
```

```
    cin >> p;                                                  //Line 9
    cout << endl << "Line 10: p(x): " << p << endl;            //Line 10

    cout << "Line 11: p(5): " << p(5) << endl << endl; //Line 11

    cin >> q;                                                  //Line 12
    cout << endl << "Line 13: q(x): " << q << endl
         << endl;                                              //Line 13

    t = p + q;                                                 //Line 14

    cout << "Line 15: p(x) + q(x): " << t << endl;             //Line 15

    cout << "Line 16: p(x) - q(x): " << p - q << endl; //Line 16

    return 0;                                                  //Line 17
}                                                              //Line 18
```

Sample Run: In this sample run, the user input is shaded.

```
The degree of this polynomial is: 7
Enter the coefficient of x^0: 0
Enter the coefficient of x^1: 1
Enter the coefficient of x^2: 4
Enter the coefficient of x^3: 0
Enter the coefficient of x^4: 0
Enter the coefficient of x^5: 0
Enter the coefficient of x^6: 0
Enter the coefficient of x^7: 6

Line 10: p(x): 1x^1 + 4x^2 + 6x^7
Line 11: p(5): 468855

The degree of this polynomial is: 3
Enter the coefficient of x^0: 1
Enter the coefficient of x^1: 2
Enter the coefficient of x^2: 0
Enter the coefficient of x^3: 3

Line 13: q(x): 1 + 2x^1 + 3x^3

Line 15: p(x) + q(x): 1 + 3x^1 + 4x^2 + 3x^3 + 6x^7
Line 16: p(x) - q(x): -1 - 1x^1 + 4x^2 - 3x^3 + 6x^7
```

QUICK REVIEW

1. Pointer variables contain the addresses of other variables as their values.
2. In C++, no name is associated with the pointer data type.
3. A pointer variable is declared using an asterisk, `*`, between the data type and the variable.

4. In C++, `&` is called the address of operator.
5. The address of operator returns the address of its operand. For example, if `p` is a pointer variable of type `int` and `num` is an `int` variable, the statement

   ```
   p = &num;
   ```

   sets the value of `p` to the address of `num`.
6. When used as a unary operator, `*` is called the dereferencing operator.
7. The memory location indicated by the value of a pointer variable is accessed by using the dereferencing operator, `*`. For example, if `p` is a pointer variable of type `int`, the statement

   ```
   *p = 25;
   ```

   sets the value of the memory location indicated by the value of `p` to `25`.
8. You can use the member access operator arrow, `->`, to access the component of an object pointed to by a pointer.
9. Pointer variables are initialized using either `0` (the integer zero), `NULL`, or the address of a variable of the same type.
10. The only integer value that can be directly assigned to a pointer variable is `0`.
11. The only arithmetic operations allowed on pointer variables are increment (`++`), decrement (`--`), addition of an integer to a pointer variable, subtraction of an integer from a pointer variable, and subtraction of a pointer from another pointer.
12. Pointer arithmetic is different from ordinary arithmetic. When an integer is added to a pointer, the value added to the value of the pointer variable is the integer times the size of the object to which the pointer is pointing. Similarly, when an integer is subtracted from a pointer, the value subtracted from the value of the pointer variable is the integer times the size of the object to which the pointer is pointing.
13. Pointer variables can be compared using relational operators. (It makes sense to compare pointers of the same type.)
14. The value of one pointer variable can be assigned to another pointer variable of the same type.
15. A variable created during program execution is called a dynamic variable.
16. The operator `new` is used to create a dynamic variable.
17. The operator `delete` is used to deallocate the memory occupied by a dynamic variable.
18. In C++, both `new` and `delete` are reserved words.
19. The operator `new` has two forms: one to create a single dynamic variable, and another to create an array of dynamic variables.
20. If `p` is a pointer of type `int`, the statement

    ```
    p = new int;
    ```

    allocates storage of type `int` somewhere in memory and stores the address of the allocated storage in `p`.

21. The operator `delete` has two forms: one to deallocate the memory occupied by a single dynamic variable, and another to deallocate the memory occupied by an array of dynamic variables.
22. If `p` is a pointer of type `int`, the statement `delete p;` deallocates the memory to which `p` points.
23. The array name is a constant pointer. It always points to the same memory location, which is the location of the first array component.
24. To create a dynamic array, the form of the `new` operator that creates an array of dynamic variables is used. For example, if `p` is a pointer of type `int`, the statement

    ```
    p = new int[10];
    ```

    creates an array of 10 components of type `int`. The base address of the array is stored in `p`. We call `p` a dynamic array.
25. Array notation can be used to access the components of a dynamic array. For example, suppose `p` is a dynamic array of 10 components. Then `p[0]` refers to the first array component, `p[1]` refers to the second array component, and so on. In particular, `p[i]` refers to the `(i + 1)`th component of the array.
26. An array created during program execution is called a dynamic array.
27. If `p` is a dynamic array, then the statement

    ```
    delete [] p;
    ```

    deallocates the memory occupied by `p`—that is, the components of `p`.
28. In a shallow copy, two or more pointers of the same type point to the same memory space; that is, they point to the same data.
29. In a deep copy, two or more pointers of the same type have their own copies of the data.
30. If a class has a destructor, the destructor automatically executes whenever a class object goes out of scope.
31. If a class has pointer data members, the built-in assignment operators provide a shallow copy of the data.
32. A copy constructor executes when an object is declared and initialized by using the value of another object, and when an object is passed by value as a parameter.
33. C++ allows a user to pass an object of a derived class to a formal parameter of the base class type.
34. The binding of virtual functions occurs at execution time, not at compile time, and is called dynamic or run-time binding.
35. In C++, virtual functions are declared using the reserved word virtual.
36. A class is called an abstract class if it contains one or more pure virtual functions.

37. Because an abstract class is *not* a complete classas—it (or its implementation file) does not contain the definitions of certain functions as you cannot create objects of that class.
38. In addition to the pure virtual functions, an abstract class can contain instance variables, constructors, and functions that are not pure virtual. However, the abstract class must provide the definitions of constructors and functions that are not pure virtual.
39. A list is a collection of elements of the same type.
40. The commonly performed operations on a list are create the list, determine whether the list is empty, determine whether the list is full, find the size of the list, destroy or clear the list, determine whether an item is the same as a given list element, insert an item in the list at the specified location, remove an item from the list at the specified location, replace an item at the specified location with another item, retrieve an item from the list from the specified location, and search the list for a given item.

EXERCISES

1. Mark the following statements as true or false.
   a. In C++, `pointer` is a reserved word.
   b. In C++, pointer variables are declared using the reserved word `pointer`.
   c. The statement `delete p;` deallocates the variable pointer `p`.
   d. The statement `delete p;` deallocates the dynamic variable to which `p` points.
   e. Given the declaration

      ```
      int list[10];
      int *p;
      ```

      the statement

      ```
      p = list;
      ```

      is valid in C++.
   f. Given the declaration

      ```
      int *p;
      ```

      the statement

      ```
      p = new int[50];
      ```

      dynamically allocates an array of 50 components of type `int`, and `p` contains the base address of the array.
   g. The address of operator returns the address and value of its operand.
   h. If `p` is a pointer variable, the statement `p = p * 2;` is valid in C++.

2. Given the declaration

```
int x;
int *p;
int *q;
```

Mark the following statements as valid or invalid. If a statement is invalid, explain why.

a. `p = q;`

b. `*p = 56;`

c. `p = x;`

d. `*p = *q;`

e. `q = &x;`

f. `*p = q;`

3. What is the output of the following C++ code?

```
int x;
int y;
int *p = &x;
int *q = &y;
*p = 35;
*q = 98;
*p = *q;
cout << x << "  " << y << endl;
cout << *p << "  " << *q << endl;
```

4. What is the output of the following C++ code?

```
int x;
int y;
int *p = &x;
int *q = &y;
x = 35;  y = 46;
p = q;
*p = 78;
cout << x << "  " << y << endl;
cout << *p << "  " << *q << endl;
```

5. Given the declaration

```
int num = 6;
int *p = &num;
```

which of the following statement(s) increment the value of `num`?

a. `p++;`

b. `(*p)++;`

c. `num++`

d. `(*num)++;`

6. What is the output of the following code?

```
int *p;
int * q;
p = new int;
q = p;
*p = 46;
*q = 39;
cout << *p << " " << *q << endl;
```

7. What is the output of the following code?

```
int *p;
int *q;
p = new int;
*p = 43;
q = p;
*q = 52;
p = new int;
*p = 78;
q = new int;
*q = *p;
cout << *p << " " << *q << endl;
```

8. What is wrong with the following code?

```
int *p;                                  //Line 1
int *q;                                  //Line 2

p = new int;                             //Line 3
*p = 43;                                 //Line 4

q = p;                                   //Line 5
*q = 52;                                 //Line 6

delete q;                                //Line 7

cout << *p << " " << *q << endl;         //Line 8
```

9. What is the output of the following code?

```
int x;
int *p;
int *q;
p = new int[10] ;
q = p;
*p = 4;

for(int j = 0; j < 10; j++)
{
    x = *p;
    p++;
    *p = x + j;
}
```

```
for (int k = 0; k < 10; k++)
{
    cout << *q << " ";
    q++;
}
cout << endl;
```

10. What is the output of the following code?

```
int *secret;

secret = new int[10];
secret[0] = 10;
for (int j = 1; j < 10; j++)
    secret[j] = secret[j -1] + 5;
for(int j = 0; j < 10; j++)
    cout << secret[j] << " ";
cout << endl;
```

11. Explain the difference between a shallow copy and a deep copy of data.

12. What is wrong with the following code?

```
int *p;                          //Line 1
int *q;                          //Line 2

p = new int [5];                 //Line 3
*p = 2;                          //Line 4

for (int i = 1; i < 5; i++)      //Line 5
    p[i] = p[i-1] + i;           //Line 6

q = p;                           //Line 7

delete [] p;                     //Line 8

for (int j = 0; j < 5; j++)      //Line 9
    cout << q[j] << " ";         //Line 10

cout << endl;                    //Line 11
```

13. What is the output of the following code?

```
int *p;
int *q;

p = new int [5];
p[0] = 5;

for (int i = 1; i < 5; i++)
    p[i] = p[i - 1] + 2 * i;

cout << "Array p: ";
for (int i = 0; i < 5; i++)
    cout << p[i] << " ";
cout << endl;
```

```
q = new int[5];

for (int i = 0; i < 5; i++)
    q[i] = p[4 - i];

cout << "Array q: ";
for (int i = 0; i < 5; i++)
    cout << q[i] << " ";

cout << endl;
```

14. What is the output of the following code?

```
int **p;

p = new int* [5];

for (int i = 0; i < 5; i++)
    p[i] = new int[3];

for (int i = 1; i < 5; i++)
    for (int j = 0; j < 3; j++)
        p[i][j] = 2 * i + j;

for (int i = 1; i < 5; i++)
{
    for (int j = 0; j < 3; j++)
        cout << p[i][j] << " ";
    cout << endl;
}
```

15. What is the purpose of the copy constructor?
16. Name three situations when a copy constructor executes.
17. Name three things that you should do for classes with pointer data members.
18. Suppose that you have the following definition of a class.

```
class dummyClass
{
public:
    void print();
    ...
private:
    int listLength;
    int *list;
    double salary;
    string name;
}
```

    a. Write the function prototype to overload the assignment operator for the `class dummyClass`.
    b. Write the definition of the function to overload the assignment operator for the `class dummyClass`.

c. Write the function prototype to include the copy constructor for the `class dummyClass`.

d. Write the definition of the copy constructor for the `class dummyClass`.

19. Suppose that you have the following classes, `classA` and `classB`:

```
class classA
{
public:
    virtual void print() const;
    void doubleNum();
    classA(int a = 0);

private:
    int x;
};

void classA::print() const
{
    cout << "ClassA x: " << x << endl;
}

void classA::doubleNum()
{
    x = 2 * x;
}

classA::classA(int a)
{
    x = a;
}

class classB: public classA
{
public:
    void print() const;
    void doubleNum();
    classB(int a = 0, int b = 0);

private:
    int y;
};

void classB::print() const
{
    classA::print();
    cout << "ClassB y: " << y << endl;
}
```

```
void classB::doubleNum()
{
    classA::doubleNum();

    y = 2 * y;
}

classB::classB(int a, int b)
       : classA(a)
{
    y = b;
}
```

What is the output of the following function `main`?

```
int main()
{
    classA *ptrA;
    classA objectA(2);

    classB objectB(3, 5);

    ptrA = &objectA;
    ptrA->doubleNum();
    ptrA->print();
    cout << endl;

    ptrA = &objectB;

    ptrA->doubleNum();
    ptrA->print();
    cout << endl;

    return 0;
}
```

20. What is the output of the function `main` of Exercise 19, if the definition of `classA` is replaced by the following definition?

```
class classA
{
public:
    virtual void print() const;
    virtual void doubleNum();
    classA(int a = 0);

private:
    int x;
};
```

21. What is the difference between compile-time binding and run-time binding?

22. Consider the following definition of the `class student`.

```
public studentType: public personType
{
public:
    void print();
    void calculateGPA();
    void setID(long id);
    void setCourses(const string c[], int noOfC);
    void setGrades(const char cG[], int noOfC);

    void getID();
    void getCourses(string c[], int noOfC);
    void getGrades(char cG[], int noOfC);
    void studentType(string fName = "", string lastName = "",
                     long id, string c[] = NULL,
                     char cG[] = NULL, int noOfC = 0);

private:
    long studentId;
    string courses[6];
    char coursesGrade[6]
    int noOfCourses;
}
```

Rewrite the definition of the `class student` so that the functions `print` and `calculateGPA` are pure `virtual` functions.

23. What is the effect of the following statements?

    a. `arrayListType<int> intList(100);`

    b. `arrayListType<string> stringList(1000);`

    c. `arrayListType<double> salesList(-10);`

24. Draw the UML diagram of the `class polynomialType`. Also show the inheritance hierarchy.

PROGRAMMING EXERCISES

1. The function `removeAt` of the `class arrayListType` removes an element from the list by shifting the elements of the list. However, if the element to be removed is at the beginning of the list and the list is fairly large, it could take a lot of computer time. Because the list elements are in no particular order, you could simply remove the element by swapping the last element of the list with the item to be removed and reducing the length of the list. Rewrite the definition of the function `removeAt` using this technique.

2. The function `remove` of the `class arrayListType` removes only the first occurrence of an element. Add the function `removeAll` to the `class arrayListType` that would remove all occurrences of a given element. Also, write the definition of the function `removeAll` and a program to test this function.

3. Add the function `min` to the `class arrayListType` to return the smallest element of the list. Also, write the definition of the function `min` and a program to test this function.
4. Add the function `max` to the `class arrayListType` to return the largest element of the list. Also, write the definition of the function `max` and a program to test this function.
5. The operators `+` and `-` are overloaded as member functions for the `class polynomialType`. Redo Programming Example Polynomial Operations so that these operators are overloaded as nonmember functions. Also write a test program to test these operators.
6. Write the definition of the function to overload the operator `*` (as a member function) for the `class polynomialType` to multiply two polynomials. Also write a test program to test the operator `*`.
7. Let $p(x) = a_0 + a_1x + \ldots + a_{n-1}x^{n-1} + a_nx^n$ be a polynomial of degree n, where a_i are real (or complex) numbers and n is a nonnegative integer. The derivative of $p(x)$, written $p'(x)$, is defined to be $p'(x) = a_1 + 2a_2x^2 + \ldots + na_nx^{n-1}$. If $p(x)$ is constant, then $p'(x) = 0$. Overload the operator $\sim$ as a member function for the `class polynomialType` so that $\sim$ returns the derivative of a polynomial.
8. The `class polynomialType` as given in the Programming Example Polynomial Operations processes polynomials with coefficients that are real numbers. Design and implement a similar class that can be used to process polynomials with coefficients as complex numbers. Your class must overload the operators `+`, `-`, `*` to perform addition, subtraction, and multiplication; and the operator `()` to evaluate a polynomial at a given complex number. Also write a program to test various operations.
9. Using classes, design an online address book to keep track of the names, addresses, phone numbers, and dates of birth of family members, close friends, and certain business associates. Your program should be able to handle a maximum of 500 entries.

    a. Define a `class`, `addressType`, that can store a street address, city, state, and zip code. Use the appropriate functions to print and store the address. Also, use constructors to automatically initialize the data members.

    b. Define a `class extPersonType` using the `class personType` (as defined in Example 1-12, Chapter 1), the `class dateType` (as designed in Programming Exercise 2 of Chapter 2), and the `class addressType`. Add a data member to this class to classify the person as a family member, friend, or business associate. Also, add a data member to store the phone number. Add (or override) the functions to print and store the appropriate information. Use constructors to automatically initialize the data members.

    c. Derive the `class addressBookType` from the `class arrayListType`, as defined in this chapter, so that an object of type `addressBookType` can store

objects of type `extPersonType`. An object of type `addressBookType` should be able to process a maximum of 500 entries. Add necessary operations to the `class addressBookType` so that the program should perform the following operations:

i. Load the data into the address book from a disk.
ii. Search for a person by last name.
iii. Print the address, phone number, and date of birth (if it exists) of a given person.
iv. Print the names of the people whose birthdays are in a given month or between two given dates.
v. Print the names of all the people having the same status, such as family, friend, or business.
vi. Print the names of all the people between two last names.

10. (**Safe Arrays**) In C++, there is no check to determine whether the array index is out of bounds. During program execution, an out-of-bound array index can cause serious problems. Also, recall that in C++ the array index starts at 0.

    Design a `class safeArray` that solves the out-of-bound array index problem and allows the user to begin the array index starting at any integer, positive or negative. Every object of type `safeArray` should be an array of type `int`. During execution, when accessing an array component, if the index is out of bounds, the program must terminate with an appropriate error message. For example,

```
safeArray list(2,13);
safeArray yourList(-5,9);
```

    In this example, `list` is an array of 11 components, the component type is `int`, and the components are `list[2]`, `list[3]`, ..., `list[12]`. Also, `yourList` is an array of 15 components, the component type is `int`, and the components are `yourList[-5]`, `yourlist[-4]`, ..., `yourList[0]`, ..., `yourList[8]`.

11. Programming Exercise 10 processes only `int` arrays. Redesign the `class safeArray` using class templates so that the class can be used in any application that requires arrays to process data.

12. Design a class to perform various matrix operations. A matrix is a set of numbers arranged in rows and columns. Therefore, every element of a matrix has a row position and a column position. If A is a matrix of 5 rows and 6 columns, we say that matrix A is of the size 5×6 and sometimes denote it as $A_{5\times6}$. Clearly, a convenient place to store a matrix is in a two-dimensional array. Two matrices can be added and subtracted if they have the same size. Suppose that $A = [a_{ij}]$ and

$B = [b_{ij}]$ are two matrices of the size $m \times n$, where a_{ij} denotes the element of A in the ith row and the jth column, and so on. The sum and difference of A and B is given by

$$A + B = [a_{ij} + b_{ij}]; \qquad A - B = [a_{ij} - b_{ij}]$$

The multiplication of A and B (A * B) is defined only if the number of columns of A are the same as the number of rows of B. If A is of the size $m \times n$ and B is of the size $n \times t$, then $A * B = [c_{ik}]$ is of the size $m \times t$ and the element c_{ik} is given by the formula

$$c_{ik} = a_{i1}b_{1k} + a_{i2}b_{2k} + \ldots + a_{in}b_{nk}$$

Design and implement a `class matrixType` that can store a matrix of any size. Overload the operators `+`, `-`, and `*` to perform the addition, subtraction, and multiplication operations, respectively, and overload the operator `<<` to output a matrix. Also, write a test program to test various operations on matrices.

13. The `class largeIntegers` in Programming Exercise 16, in Chapter 2, is designed to process large integers of at most 100 digits. Using dynamic arrays, redesign this class so that integers of any digits can be added and/or subtracted. Also overload the multiplication operator to multiply large integers.